A Unified Framework for Function-Level Vulnerability Detection and Explanation in Smart Contracts

NGOC MINH NGUYEN and LE MINH NGUYEN*, Nguyen Lab, Division of Advanced Science and Technology, Japan Advanced Institute of Science and Technology, Japan

Smart contracts enable decentralized applications, yet their immutability makes security vulnerabilities catastrophic, often leading to irrecoverable financial losses and systemic risks. Existing machine learning approaches typically operate at the coarse contract level, failing to localize issues to specific functions or provide interpretable remediation guidance for developers. To address these persistent limitations, we introduce a unified framework combining a novel Hierarchical Cross-Attention Subgraph Neural Network (HCA-SGNN) for detection with Large Language Models (LLMs) for explanation. For detection, HCA-SGNN processes functions as subgraphs, explicitly modeling control- and data-flow dependencies while capturing cross-function interactions to pinpoint localized risks. To enable robust training, we develop an AST-driven injection system that generates stealthy, context-aware vulnerabilities through template-based synthesis. Uniquely, this system ensures collision-free variable naming and version-specific compatibility, producing a comprehensive, function-level annotated dataset covering diverse classes such as reentrancy, integer overflows, and access control flaws. Complementing detection, we generate structured, human-readable justifications using synthetic data and chain-of-thought prompting. These explanations detail the vulnerability type, affected area, root cause, and actionable mitigation strategies, effectively bridging the gap between automated detection and practical insight. Together, these components form a comprehensive framework that significantly enhances both the technical accuracy and operational usability of smart contract analysis.

CCS Concepts: • **Computing methodologies** → **Supervised learning**; **Natural language generation**; • **Computer systems organization** → **Neural networks**; • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: LLM, Supervised Learning, Vulnerability Detection, Deep Learning, Smart Contract, Graph Neural Network

1 Introduction

The emergence of blockchain technology has enabled a new paradigm of decentralized applications, with smart contracts—self-executing programs deployed on platforms such as Ethereum—serving as the backbone of trustless computation [Ming Jie et al. 2024]. These contracts automate financial transactions, digital asset exchanges, governance protocols, and decentralized finance (DeFi) operations, eliminating the need for intermediaries. Written primarily in domain-specific languages like Solidity, smart contracts are immutable once deployed [Wei et al. 2025], which means that any coding flaw becomes permanent and immediately exploitable. Consequently, security vulnerabilities in smart contracts have led to high-profile exploits, resulting in significant financial and reputational damage within the blockchain ecosystem.

*Corresponding author.

Authors' Contact Information: Ngoc Minh Nguyen, minh.nn@jaist.ac.jp; Le Minh Nguyen, nguyenml@jaist.ac.jp, Nguyen Lab, Division of Advanced Science and Technology, Japan Advanced Institute of Science and Technology, Ishikawa, Japan.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/8-ART

https://doi.org/10.1145/3840386

Just Accepted

Although static analysis and machine learning–based techniques have advanced smart contract vulnerability detection [Gurjar and Chandavarkar 2024; Iuliano and Nucci 2024; S. J. Wang et al. 2024], three persistent limitations remain. First, while some tools—such as Securify [ETH SRI (Secure, Reliable, and Intelligent Systems Lab) 2025] and Slither [Crytic 2025]—can flag specific instructions or code locations, they do not perform *function-level* vulnerability analysis. Since developers reason about code at the function level—functions are the natural unit of design, review, and testing—instruction-level or bytecode-level outputs still require manual effort to map alerts to the responsible function and its broader context. This granularity mismatch limits the practical utility of automated detection. Second, existing tools rarely provide meaningful explanations; outputs are typically reduced to binary labels, line markers, or pattern-match warnings (e.g., "Reentrancy in Contract.withdraw()"), with little semantic reasoning about why the code is vulnerable, what the consequences are, or how to remediate the issue. Without such interpretability, tools fall short in guiding developers toward effective fixes. Third, there are currently no large-scale datasets that provide function-level vulnerability annotations for the Solidity language. Existing resources are typically curated at the contract level, which obscures the fine-grained vulnerability patterns that occur within individual functions. This lack of function-level data limits the ability of models to learn localized representations of vulnerabilities and constrains research on precise detection and explanation.

To address the challenges of detecting and explaining vulnerabilities in smart contracts, our methodology is structured around three complementary components: dataset construction, function-level detection, and explanation generation. Each component directly aligns with our research objectives and contributes to the development of a robust and interpretable analysis framework. First, we develop an AST-based injection system that generates stealthy, context-aware vulnerabilities for large-scale dataset construction. Unlike existing snippet-based approaches such as SolidiFI [Ghaleb and Pattabiraman 2020], which use hard-coded code patterns with generic variable names, our system dynamically synthesizes vulnerabilities using template-based generation with collision-free naming and version-specific adaptations. The system implements three algorithms: (1) AST-driven injection point detection that identifies semantically valid locations through state variable analysis, (2) collision-free variable name synthesis from realistic, type-appropriate pools, and (3) version-aware template filtering that ensures compatibility across Solidity 0.4.x–0.8.x. This approach achieves 96% compilation success and produces function-level annotations including vulnerability class, affected code regions, and synthesized variable metadata. Second, we introduce a novel Hierarchical Cross-Attention Subgraph GNN (HCA-SGNN) designed to perform fine-grained vulnerability detection. By restricting message passing to subgraphs corresponding to individual functions, our model captures localized patterns of unsafe operations while maintaining awareness of broader structural context through dual-channel message passing with gated fusion and inter-function cross-attention. Training this architecture on our synthetic dataset allows us to classify vulnerabilities at the function level with improved resolution compared to prior whole-contract approaches. Finally, we address the challenge of interpretability by developing a high-quality explanation generation system. Leveraging Chain-of-Thought (CoT) prompting with GPT-4o, we generate structured, step-by-step explanations of vulnerabilities. These explanations are subsequently distilled into a smaller, specialized model, ensuring both scalability and accessibility. This dual-stage approach combines the reasoning capabilities of large models with the efficiency of compact models, enabling practical deployment. Together, these components form a cohesive methodology: AST-based synthesis of datasets to enable learning, HCA-SGNN for precise detection, and CoT-based explanation generation for interpretability.

In summary, this paper makes the following contributions:

(1) **Function-level subgraph decomposition.** We propose the first subgraph neural network architecture for *function-level* smart contract vulnerability classification. Unlike all prior graph-based detectors [Z. Liu et al. 2021; Yuan Zhuang et al. 2020], which treat the entire contract graph as a single instance and

Just Accepted

produce a contract-level label via global pooling, HCA-SGNN decomposes each contract into function-level subgraphs and produces per-function predictions that align with developer workflows.

(2) **Heterogeneous, typed-edge graph construction.** We enrich the standard AST with semantic edges—data-flow references (REF), explicit control-flow (CFG_NEXT, CFG_TRUE, CFG_FALSE), inter-function calls (CALL), inheritance links (INHERIT), and guard edges (GUARD)—producing a heterogeneous graph with 86 node types and 8 semantically distinct edge types. This enables the model to distinguish structurally and semantically different relationships during message passing—a significant departure from the homogeneous, untyped-edge graphs used by prior approaches.

(3) **Dual-channel message passing with gated fusion and inter-function cross-attention.** HCA-SGNN maintains two parallel convolution channels—an intra-function *local* channel and a contract-wide *global* channel—fused via a per-node learned gate. An inter-function cross-attention module further enables detection of coupled vulnerabilities (e.g., reentrancy) that span multiple functions. Our ablation study confirms that the dual-channel design alone accounts for a 23.6 percentage-point improvement in function-level F1.

(4) **AST-driven synthetic data generation.** We develop a novel AST-based injection system that generates context-aware vulnerabilities through template-based synthesis with collision-free variable naming and Solidity version-specific adaptations, producing the first large-scale, function-level annotated dataset for smart contract vulnerability detection (4,208 contracts, 118,126 functions).

(5) **Integrated explanation generation.** We combine Chain-of-Thought prompting with knowledge distillation to produce structured, human-readable explanations of detected vulnerabilities, bridging the interpretability gap left by existing detection-only approaches.

2 Backgrounds and Related Work

2.1 Vulnerability Detection for Smart Contracts

2.1.1 Static Analysis Tools. Static analysis has been a foundational approach for detecting vulnerabilities in smart contracts, particularly those written in the Solidity programming language. These tools typically analyze source code or compiled bytecode without executing the contract, aiming to uncover potential vulnerabilities such as reentrancy, arithmetic overflows, or access control violations before deployment. Over the years, several prominent tools have been developed, each employing distinct analysis techniques and offering various levels of precision, coverage, and usability. **Mythril** [ConsenSys Diligence 2025] is among the earliest and most widely used symbolic execution engines for Ethereum smart contracts. It translates bytecode into an intermediate representation and explores feasible execution paths using constraint solvers. Mythril can detect a range of common vulnerabilities, such as reentrancy and integer arithmetic bugs, by simulating different environmental conditions and control errors. **Oyente** [Enzyme Finance (formerly Oyente) 2025], one of the first vulnerability detectors for Ethereum, introduced a modular pipeline combining symbolic execution with control-flow graph (CFG) construction to detect specific vulnerability patterns. Oyente was instrumental in demonstrating the feasibility of static analysis for smart contract security, but its relatively limited set of vulnerability checks and high false-positive rates restricted its adoption in production environments. **Securify** [ETH SRI (Secure, Reliable, and Intelligent Systems Lab) 2025] performs static analysis on the EVM bytecode of smart contracts to extract key semantic properties, such as control-flow and data-flow information. This process is fully automated using Soufflé, a high-performance Datalog solver. Based on the derived facts, Securify then verifies whether the contract contains security violations or if its security-critical instructions comply with best practices. **Slither** [Crytic 2025] stands out for its developer-oriented design and extensibility. Built on a custom static analysis framework, it translates Solidity smart contracts into an intermediate representation called SlithIR. SlithIR uses Static Single Assignment (SSA) form and a reduced instruction set to ease implementation of analyses while preserving

Just Accepted

semantic information that would be lost in transforming Solidity to bytecode. Slither enables the application of commonly used program analysis techniques, such as dataflow and taint tracking.

Beyond detection-oriented static tools, several complementary approaches have been proposed. **Sereum** [Rodler et al. 2018] introduced a runtime monitoring system that protects deployed contracts against re-entrancy attacks by intercepting execution flows at the EVM level. Unlike offline analysis tools, Sereum validates storage-update patterns during execution to detect and prevent cross-function, delegated, and create-based re-entrancy in real time. While effective as a prevention mechanism, Sereum is limited to reentrancy and operates at the bytecode/runtime level, making it complementary to source-level detection approaches like ours. **ReEP** [Z. Wang et al. 2025] takes a meta-analysis approach, integrating results from multiple existing detection tools and applying post-hoc verification to reduce false positives. By unifying eight reentrancy detectors, ReEP increased average precision from 0.5% to 73% without sacrificing recall. However, ReEP does not introduce new detection capabilities; its effectiveness depends on the union of its constituent tools, and it inherits their collective coverage limitations.

While these tools have proven effective in detecting certain classes of vulnerabilities, they share several limitations. Although some tools—notably Securify and Slither—can localize issues to specific instructions or code segments, none provide systematic *function-level* vulnerability classification. Securify classifies individual instructions as "violation," "warning," or "safe" against predefined security patterns, and Slither reports detector-level findings with line references; however, these outputs still require developers to reconstruct which function is affected and to reason about cross-function dependencies. Furthermore, their reliance on handcrafted rules or path exploration heuristics makes it difficult to adapt to previously unseen vulnerability types or code patterns. Finally, most static tools offer limited explanatory capabilities; alerts are typically reported as pattern-match warnings with minimal contextual information, lacking structured explanations of root causes, impact, and remediation strategies.

2.1.2 Machine Learning Approaches for Detection. To mitigate the drawbacks of rule-based methods, machine learning (ML) has gained traction as a complement to static analysis for detecting smart contract vulnerabilities [Gurjar and Chandavarkar 2024; Iuliano and Nucci 2024; Wei et al. 2025]. Unlike rule-based or symbolic execution approaches, ML models can learn vulnerability patterns from labeled data, improving generalization to unseen code and reducing dependence on handcrafted heuristics. Existing methods can be broadly categorized into three main types: feature-based models, sequence-based models, and representation learning using graphs or embeddings. Feature-based approaches represent early attempts to apply ML in this domain. These methods rely on manually engineered features derived from source code, bytecode, or abstract syntax trees (ASTs). For example, Zhu et al.[J. Zhu et al. 2023] trained LSTM models on opcode sequences labeled via the SODA plugin, while other studies leveraged external call counts or control-flow metrics[Gurjar and Chandavarkar 2024; Iuliano and Nucci 2024]. Although effective in some cases, their performance is constrained by the quality of the feature engineering and their inability to capture deep semantic dependencies. Sequence-based deep learning methods improve flexibility by learning directly from tokenized code or opcode streams. For instance, ContractWard [W. Wang et al. 2021] applied bigram features and gradient boosting ensembles to achieve efficient contract-level detection. However, such models often fail to preserve structural relationships, such as control-flow or data dependencies, which are crucial for accurate vulnerability reasoning. Graph-based models address this limitation by encoding contracts as structural representations such as CFGs, PDGs, or ASTs. Zhuang et al. [Yuan Zhuang et al. 2020] used graph neural networks (GNNs), including DR-GCN and Temporal Message Propagation, to learn relational patterns for vulnerability detection. These methods achieve strong performance but remain restricted to contract-level classification, offering little guidance for localizing vulnerabilities to specific functions. Liu et al. [Z. Liu et al. 2021] proposed combining GNNs with expert-defined security patterns, casting control- and data-flow semantics into a contract graph and applying temporal message propagation for feature extraction. While this

Just Accepted

approach achieves strong detection accuracy (89.15% for reentrancy), it remains a contract-level method and does not provide function-level localization. More recently, Osei et al. [Osei et al. 2025] introduced AWDNN, an attention-based wide and deep neural network for reentrancy detection that uses code vectorization to capture semantic features of smart contract fragments. Although AWDNN demonstrates competitive performance, it focuses exclusively on reentrancy and relies on sequential code representations rather than graph structures, limiting its ability to capture control- and data-flow dependencies.

Beyond detection, recent studies have explored code embeddings and multitask learning to improve generalization and interpretability. Approaches such as Code2Vec [Alon et al. 2018], CodeBERT [Feng et al. 2020], and GraphCodeBERT [Guo, Ren, et al. 2021] extract semantic representations of functions and data flows, enabling downstream classification tasks. Two recent lines of work explore complementary modalities. *Bytecode-level transformer models*: SmartBugBert [Bu et al. 2025] combines BERT with control-flow graph analysis on *decompiled opcode sequences* to detect four vulnerability types (transaction-ordering, access control, self-destruct, timestamp dependency) directly from EVM bytecode, achieving 91% F1 on a corpus of 6,157 contracts. Because SmartBugBert operates on bytecode rather than source code, it targets a complementary deployment scenario—contracts whose source is not publicly available—and uses a different vulnerability taxonomy, making direct experimental comparison with source-code-based methods methodologically inappropriate without a shared evaluation protocol (see Section 4.2.2). *LLM-based prompting frameworks*: SmartVD [Alam et al. 2024] benchmarks open-source LLMs (CodeLlama, Llama2, CodeT5, Falcon) and fine-tuned closed-source models (GPT-3.5 Turbo, GPT-4o Mini) for multi-class vulnerability classification against the OWASP Top Ten taxonomy, using zero-shot, few-shot, and chain-of-thought prompting on their proprietary VulSmart dataset. While SmartVD demonstrates strong results with fine-tuned GPT models (99% accuracy), its evaluation protocol differs substantially from ours: it uses a different dataset, a different vulnerability taxonomy, and multi-class rather than binary classification, precluding direct numerical comparison (see Section 4.2.2).

While the above methods introduce valuable inductive biases, they often flatten or oversimplify hierarchical structures, limiting their effectiveness for fine-grained vulnerability reasoning. Despite these advancements, three key challenges remain: (1) a lack of fine-grained datasets with function-level annotations, (2) limited interpretability of ML models, which essentially act as black boxes, and (3) insufficient integration with developer workflows—even tools that provide sub-contract-level outputs still lack function-level classification, structured semantic explanations, and actionable remediation guidance within a unified framework.

Positioning of HCA-SGNN relative to prior graph-based approaches. Our work departs from the standard graph-based paradigm along five dimensions. (i) *Granularity*: all prior GNN-based detectors [Z. Liu et al. 2021; Yuan Zhuang et al. 2020] classify at the contract level; HCA-SGNN is the first to decompose contract graphs into function-level subgraphs and predict per-function labels. (ii) *Graph heterogeneity*: existing methods use homogeneous graphs with undifferentiated edges, whereas our graph construction enriches the AST with semantic edges (data-flow references, explicit control-flow, inter-function calls, inheritance, and guard links), producing a heterogeneous graph with 8 semantically distinct edge types and 86 node types that enables edge-type-aware convolution. (iii) *Dual-channel architecture*: prior approaches apply a single message-passing channel over the entire contract graph, which mixes signals from unrelated functions; HCA-SGNN separates intra-function (local) and inter-function (global) channels and fuses them via a learned per-node gate, yielding a 23.6 pp improvement in function-level F1 over the single-channel baseline (Section 3.3.3). (iv) *Cross-attention*: HCA-SGNN introduces an inter-function cross-attention module that explicitly captures coupled vulnerability patterns (e.g., reentrancy), which no prior graph-based detector models. (v) *Integrated pipeline*: beyond detection, our framework couples HCA-SGNN with AST-driven synthetic data generation and Chain-of-Thought explanation generation, providing a unified detect-and-explain workflow absent from all prior graph-based approaches.

Just Accepted

2.1.3 Synthetic Vulnerability Generation. To address the scarcity of labeled training data, researchers have developed synthetic vulnerability injection approaches. SolidiFI [Ghaleb and Pattabiraman 2020] pioneered this direction by introducing a repository of hard-coded vulnerability snippets organized by vulnerability type. While SolidiFI automatically identifies potential injection locations through AST analysis, it relies on pre-written code patterns with generic variable names (e.g., `balances_re_ent3`), which can cause compilation failures due to identifier collisions or version incompatibilities. Moreover, SolidiFI's snippets are fixed patterns that do not adapt to the surrounding code context, limiting their stealthiness and realism. Other approaches have explored mutation-based techniques, where existing vulnerable contracts are modified to generate variants. However, these methods inherit the limitations of their seed datasets and struggle to produce diverse vulnerability patterns. Recent work has also investigated program synthesis and template-based generation, but most efforts focus on simple bug patterns rather than complex, multi-statement vulnerabilities like reentrancy. Our work addresses these limitations by introducing an AST-driven injection system that dynamically synthesizes context-aware vulnerabilities. Unlike snippet-based approaches, our system uses templates with placeholders that are instantiated using collision-free variable names selected from realistic, type-appropriate pools. The system implements version-aware filtering to ensure compatibility across Solidity versions and employs semantic analysis to identify valid injection points based on state variable assignments and function signatures. This approach achieves high compilation success rates while maintaining the diversity and realism necessary for effective model training.

3 Methodology

3.1 Problem Statement

Let $\mathcal{C}$ denote a smart contract composed of a set of functions $\{f_1, f_2, \ldots, f_n\}$. Each function f_i can be represented as a program graph $G_i = (V_i, E_i)$, where nodes V_i correspond to syntax or semantic entities (e.g., statements, variables), and edges E_i represent structural relationships such as control flow, data flow, or call dependencies.

We define three core tasks:

1. Synthetic Function-level Vulnerable Contract Generation. Given:

- A raw corpus of verified contracts $C = \{c_1, ..., c_n\}$, where each c is an Etherscan-verified Solidity smart contract.
- A set of vulnerability templates $\mathcal{T}_{vuln} = \{t_1, ..., t_m\}$, where each template t_j specifies a vulnerability pattern (e.g., reentrancy, arithmetic overflow)

The goal is to produce a labeled synthetic dataset $C' = \{c'_1, ..., c'_k\}$ where each c'_i is derived from some clean contract $c \in C$ by dynamically generating and inserting a vulnerability instance at AST-identified location p.

2. Function-Level Vulnerability Detection. Given a function-level graph G_i, the goal is to learn a function-level classifier

$$\mathcal{F}_\theta : G_i \rightarrow y_i \in \{0, 1\}$$

where $y_i = 1$ indicates that function f_i is vulnerable, and 0 otherwise. The model $\mathcal{F}_\theta$ is parameterized (e.g., by HCA-SGNN) and trained on a dataset $\mathcal{D} = \{(G_i, y_i)\}_{i=1}^{N}$, where N is the number of labeled functions.

3. Vulnerability Explanation Generation. For functions where $y_i = 1$, the goal is to generate a natural language explanation $e_i = \mathcal{G}_\phi(G_i)$, describing:

- Bug Name: the vulnerability type,
- Affected Area: vulnerability localization,
- Description: description of the vulnerability,
- Impact Assessment: the severity of the vulnerability,
- Mitigations: potential fixes or mitigations.

Just Accepted

3.2 AST-Based Vulnerability Injector

3.2.1 Data Collection and Preprocessing. We start with a corpus of real-world, open-source smart contracts collected from Etherscan[1] - a widely used Ethereum block explorer and repository for verified Solidity contracts. To collect the source codes, we utilized Etherscan's developer API. The process includes the following steps:

(1) **API Key Registration:** First, we register an account to obtain an API key for authenticated requests to the Etherscan API.
(2) **Address Collection:** We collect contract addresses either by parsing newly deployed contracts block-by-block or by downloading public datasets like SmartBugs, which already include known contract addresses.
(3) **Contract Source Retrieval:** For each address, we invoke the 'getsourcecode' endpoint:

```
https://api.etherscan.io/api?module=contract
    &action=getsourcecode
    &address=<CONTRACT_ADDRESS>
    &apikey=<API_KEY>
```

The response includes JSON-formatted fields such as 'SourceCode', 'ABI', 'ContractName', 'CompilerVersion', and 'OptimizationUsed'.
(4) **Filtering for Solidity Contracts:** We retain only those contracts whose 'SourceCode' is non-empty and 'CompilerVersion' begins with `v0.4`, `v0.5`, `v0.6`, `v0.7`, or `v0.8`, ensuring compatibility with modern static analysis tools and injection scripts.
(5) **Source Code Preprocessing:** Retrieved contracts are stored in structured directories with filenames matching the contract address. Each file is normalized by removing Etherscan wrapping artifacts, resolving imports if possible, and verifying that the code compiles with `solc`.

Using this method, we collect and preprocess over 2,000 verified contracts. For this study, we focus on 2,188 contracts curated from the SmartBugs dataset [Durieux et al. 2020], which provides on-chain verified Solidity contracts with metadata, ensuring reproducibility. To ensure the reliability of downstream vulnerability injection and label assignment, we rigorously filter out contracts that may contain latent or pre-existing vulnerabilities. We adopt a consensus-based filtering approach by applying three widely used static analysis tools:

- Slither: a static analysis framework for Solidity that performs vulnerability detection, optimization advice, and code metrics extraction. It utilizes an intermediate representation of Solidity known as SlithIR.
- Mythril: a symbolic execution engine for EVM bytecode that detects security issues such as integer overflows, callstack depth issues, and unguarded call instructions.
- Oyente: one of the earliest tools for symbolic execution of smart contracts. Though less maintained, it still detects reentrancy and transaction-ordering bugs.

Subsequently, we follow strict filtering criteria and adhere to output formatting standards. To form a high-confidence clean set $C = \{c_1, c_2, \ldots, c_n\}$, we only retain contracts that:

- Successfully compile using `solc`;
- Are not flagged by any of the three tools above;
- Have valid source code with sufficient function-level granularity (i.e., >2 function definitions). This minimum-function threshold serves two purposes: (1) it ensures that each contract provides at least one clean function alongside the injection target, allowing the model to learn the *contrast* between vulnerable and benign functions within the same contract context; and (2) it guarantees a meaningful graph structure for the dual-channel GNN architecture, which requires both intra-function and inter-function edges to

[1] https://etherscan.io/

Just Accepted

operate effectively. We note that this filter is applied only to the *injection corpus* (i.e., the source contracts used to synthesize training data); it does *not* restrict the contracts on which the trained model can be evaluated or deployed. As demonstrated in the real-world evaluation (Section 4.2.2), HCA-SGNN is evaluated on an unrestricted benchmark that includes single-function and small contracts (minimum 1 function, $Q_1 = 3$), and achieves strong generalization on these structures.

Each clean contract c_i is stored with metadata:

- `address`: Ethereum address of the deployed contract.
- `name`: Contract name (from AST).
- `source_path`: Path to the cleaned Solidity source file.
- `compiler_version`: Extracted from Etherscan metadata.

These contracts form the basis of our injection dataset pipeline. The filtering process is deterministic and reproducible, and all scripts are made available with logging at each stage for future auditing and replication. These clean contracts serve as the input to our AST-based vulnerability injection system, described next.

3.2.2 Injection System Architecture. To comprehensively model the vulnerability landscape in smart contracts, our injection system distinguishes between two fundamental vulnerability classes based on their spatial characteristics: **point vulnerabilities**, which manifest within a single function, and **coupled vulnerabilities**, which arise from interactions between multiple functions. This distinction is motivated by the observation that many real-world exploits—particularly front-running attacks and denial-of-service conditions—emerge not from isolated code defects but from unintended interactions between contract components. Our dual-mode architecture enables the synthesis of both localized patterns (e.g., arithmetic overflow within a transfer function) and systemic patterns (e.g., transaction ordering dependence across setter-executor function pairs).

Supported Vulnerability Categories. Our injection framework targets the following eight vulnerability categories, selected based on their prevalence in real-world exploits and coverage in established taxonomies (DASP Top-10, SWC Registry). For each category, we design context-aware templates that synthesize realistic vulnerable code patterns:

(1) **Access Control (`ACCESS_CONTROL`):** Access control vulnerabilities result from missing or improperly implemented authorization checks. The injected snippets often remove `require(msg.sender == owner)` or introduce unguarded state-modifying logic (e.g., transferring funds or modifying critical variables). When injected, they enable unauthorized users to invoke privileged functions, compromising the global integrity of the contract and breaching assumed trust boundaries. This compromises functions like `mint`, `updateConfig`, or `selfDestruct`, leading to privilege escalation and unauthorized control.
(2) **Integer Overflow (`OVERFLOW`, `UNDERFLOW`):** Integer overflow vulnerabilities occur when arithmetic operations exceed the maximum value representable by a data type, causing the value to wrap around to zero or a small number. Underflow occurs conversely when subtracting from zero or a small value. These vulnerabilities are particularly dangerous in Solidity versions prior to 0.8.0, which lack built-in overflow protection. Our templates inject patterns such as `balances[msg.sender] += amount` without SafeMath guards. Post-injection, these bugs enable attackers to manipulate token balances, bypass payment validations, or cause allocation errors. We implement 6 overflow templates and 2 underflow templates covering both mapping-based and direct variable arithmetic patterns.
(3) **Denial of Service (`DOS`):** DoS vulnerabilities occur when a contract's logic can be blocked or indefinitely stalled. Common patterns include looping over unbounded storage arrays or failing to handle external call failures. Injected snippets emulate this by, for example, inserting unbounded loops over dynamic data structures or storage-intensive logic inside frequently called functions. Once injected, these vulnerabilities

Just Accepted

impact availability, rendering functions non-executable under certain gas limits and hindering business logic, particularly during large-scale operations such as token distribution.

(4) **Transaction Order Dependence (`TOD`):** Transaction Order Dependence (also known as front-running) vulnerabilities arise when the outcome of a transaction depends on the order in which it is executed relative to other transactions. Our coupled injection templates create scenarios where a setter function updates a critical state variable (e.g., a recipient address) that is subsequently used by an executor function (e.g., to transfer funds). An attacker who observes a pending setter transaction can front-run it by submitting their own transaction with a higher gas price, thereby redirecting funds or manipulating contract state. We implement 7 version-specific TOD templates covering `transfer`, `send`, and `call` patterns across Solidity 0.4.x–0.8.x.

(5) **Reentrancy (`REENTRANCY`):** Reentrancy is introduced by inserting unsafe low-level calls (e.g., `recipient.call.value(amount)("")`) before updating the contract's internal state. Upon injection, such snippets enable external contracts to recursively invoke the same function before the original call completes, resulting in duplicated withdrawals or corrupted balances. The injected vulnerability affects all state-manipulating logic sharing the same storage and fundamentally violates the atomicity assumptions of contract execution.

(6) **Time Manipulation (`TIMESTAMP`):** This vulnerability arises when critical contract logic depends on timestamps or block numbers that can be influenced by miners. Injected code patterns include `require(now < unlockTime)` or `if (block.timestamp % 10 == 0)`. Once injected, these enable adversaries to manipulate time-based access control, locking mechanisms, or randomness seeds. As a result, the injected contract may permit early withdrawals, extend auctions unfairly, or bypass timed constraints, thus undermining time-sensitive logic across the contract. We implement 5 point injection templates and 2 coupled injection templates for timestamp-related vulnerabilities.

(7) **Unchecked External Calls (`UNCHECKED_SEND`, `UNHANDLED_EXCEPTION`):** This category encompasses two related vulnerability types. `UNCHECKED_SEND` templates inject `send()` calls without checking the boolean return value (e.g., `recipient.send(amount)` without verifying success). `UNHANDLED_EXCEPTION` templates inject low-level `call` operations that silently fail (e.g., `addr.call.value(x)("")` without a success check). Post-injection, the contract continues execution under the false assumption that the call succeeded, causing silent failures. These bugs often impact fund transfers, callback mechanisms, or event-triggered chains, breaking error-handling semantics and causing inconsistent state transitions or silent loss of value. We implement 4 `UNCHECKED_SEND` templates and 4 `UNHANDLED_EXCEPTION` templates covering Solidity 0.4.x through 0.8.x syntax variations.

(8) **Transaction Origin Authentication (`TX_ORIGIN`):** This vulnerability arises from using `tx.origin` for authentication instead of `msg.sender`. While `msg.sender` returns the immediate caller, `tx.origin` returns the original external account that initiated the transaction chain. Attackers can exploit this by creating intermediate contracts that call the vulnerable contract on behalf of a legitimate user who interacts with the attacker's contract. Our templates inject patterns such as `require(tx.origin == owner)` or conditional checks using `tx.origin` for authorization. We implement 5 `TX_ORIGIN` templates covering different authorization patterns and Solidity version syntaxes.

In total, the point injection framework comprises 36 templates spanning seven vulnerability categories: reentrancy (10 templates covering `call.value`, `send`, `delegatecall`, and `create`-based attack vectors—the latter two address advanced variants identified by Sereum [Rodler et al. 2018]), integer overflow (6 templates including the batch transfer pattern from the BECToken incident), integer underflow (2 templates), transaction origin authentication (5 templates), unchecked send (4 templates), unhandled exceptions (4 templates), and timestamp dependence (5 templates). The coupled injection framework contributes an additional 23 templates spanning six vulnerability categories: transaction order dependence (7 templates), cross-function reentrancy (9

Just Accepted

templates), denial of service (3 templates), timestamp dependence (2 templates), access control (1 template), and arithmetic overflow in time-lock contexts (1 template).

Injection Pipeline Overview. The injection process operates on the filtered corpus of clean contracts C and the vulnerability templates described above. We follow a systematic AST-based pipeline to transform clean contracts into a labeled dataset $C' = \{c'_1, c'_2, \ldots, c'_k\}$, where each c'_i contains exactly one injected vulnerability along with comprehensive metadata. The overall process is depicted in Figure 1 and formalized in Algorithm 1.

For each contract $c \in C_{\text{clean}}$, the pipeline first flattens multi-file contracts using `slither-flat` under the `OneFile` strategy, determining the root file by analyzing the import hierarchy. The flattened source is then compiled using `solcx` to generate its AST, from which the system identifies injection points—statement-level nodes within functions satisfying visibility, mutability, and parameter-type constraints. Each injection point p is tagged with its source location via the `src` field (formatted as `start:length:file_index`), enabling precise source-code rewriting. After payload insertion at the calculated byte offset, the modified contract c' is recompiled; if compilation fails, the contract is discarded and alternative templates or locations are tried. Successfully compiled contracts are annotated with vulnerability type, template name, injection function, byte offset ranges, line number ranges, and synthesized variable names, stored in both CSV and JSONL formats.

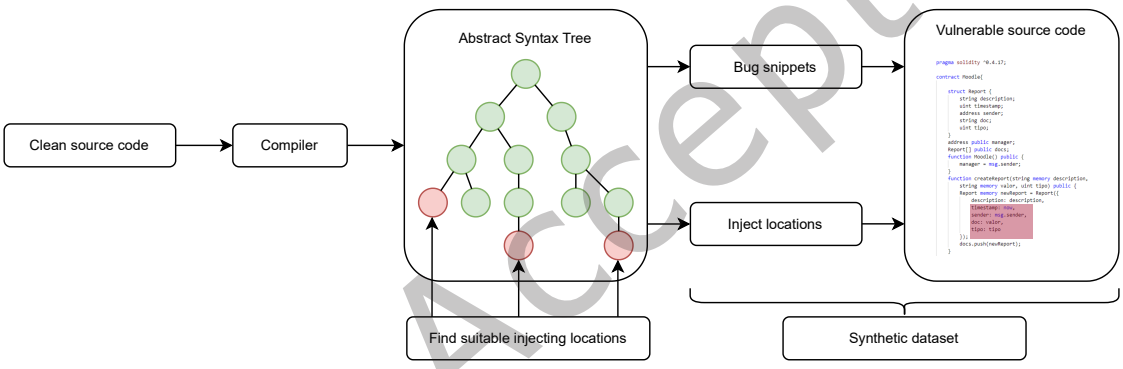


Fig. 1. AST-based Vulnerability Injection Pipeline

Point Injection. Point injection synthesizes intra-function vulnerabilities by inserting vulnerable code segments within individual function bodies. The design addresses two key challenges: *semantic validity*—ensuring that injected code compiles and executes within the host contract's context—and *stealthiness*—producing code that resembles legitimate business logic rather than obvious test patterns.

The injection process operates through three coordinated phases. First, the system performs *location analysis* by traversing the contract's AST to identify functions suitable for injection. Candidate functions must be externally callable (public or external visibility) and capable of modifying contract state. For reentrancy vulnerabilities specifically, the system further restricts candidates to functions containing state variable assignments, as these represent the critical update operations that must be protected against recursive invocations. Each candidate location is annotated with contextual metadata—parameter types, payability status, and state mutability—enabling subsequent template matching.

Second, the system performs *version-aware template selection.* Each vulnerability template specifies explicit Solidity version bounds; for example, integer overflow templates are restricted to versions prior to 0.8.0, where arithmetic operations lack built-in overflow protection. Templates also declare contextual requirements: some

Just Accepted

Algorithm 1: AST-based Vulnerability Injection for Creating Labeled Smart Contract Dataset

Input: $C = \{c_1, c_2, \ldots, c_n\}$: Set of clean smart contracts
$\mathcal{T}_{vuln}$: Set of 59 vulnerability templates (36 point + 23 coupled) with version constraints
Static analysis tools $\mathcal{T}$: e.g., Slither, Mythril, Oyente
Output: $C' = \{c'_1, c'_2, \ldots, c'_k\}$: Set of synthetic vulnerable contracts with function-level labels

Clean Contract Filtering:
$C_{\text{clean}} \leftarrow \emptyset$
foreach $c \in C$ **do**
  if `is_vulnerable`$(\mathcal{T}, c) =$ ***false*** **then**
    add c to C_{clean}

AST-based Vulnerability Injection:
$C' \leftarrow \emptyset$ `// Injected and labeled contracts`
foreach $c \in C_{clean}$ **do**
  extract AST from c using `solc --ast-compact-json`
  $sol_version \leftarrow$ `detect_version`(c)
  $functions \leftarrow$ `find_functions`(AST)
  `// Filter compatible templates based on Solidity version`
  $\mathcal{T}_{compat} \leftarrow \{t \in \mathcal{T}_{vuln} \mid t.min_version \leq sol_version \leq t.max_version\}$
  foreach $t \in \mathcal{T}_{compat}$ **do**
    foreach $f \in functions$ **do**
      `// Identify injection points via AST analysis`
      $injection_points \leftarrow$ `find_injection_locations`(f, t, AST)
      foreach $p \in injection_points$ **do**
        `// Generate context-aware payload`
        $identifiers \leftarrow$ `extract_all_identifiers`(AST)
        $payload \leftarrow$ `generate_payload`$(t, p, identifiers, sol_version)$
        `// Insert vulnerability at calculated byte offset`
        $c' \leftarrow$ `inject_at_offset`$(c, p.offset, payload)$
        if `compile`$(c') =$ ***success*** **then**
          label c' with $(t.vuln_type, f.name, p.line_range, payload.metadata)$
          add c' to C'
return C' `// Final labeled vulnerable dataset`

patterns require address-typed parameters (for value transfer), others require uint parameters (for arithmetic operations), and still others require state-modifying capabilities. Only templates satisfying all constraints for a given location proceed to the generation phase.

Third, the system performs *context-aware code generation*. Rather than inserting static code snippets, the generator dynamically synthesizes payloads that integrate with the host contract. This involves two sub-processes: (a) *collision-free variable naming*, where new identifiers are selected from domain-specific pools (financial terms like `deposits`, `allocations`; address holders like `recipient`, `beneficiary`) while avoiding conflicts with existing

Just Accepted

contract identifiers; and (b) *parameter binding*, where template placeholders are instantiated with actual function parameter names extracted from the AST. This approach eliminates the generic naming conventions of prior tools (e.g., SolidiFI's `balances_re_ent3` [Ghaleb and Pattabiraman 2020]) that frequently cause compilation failures and produce obviously synthetic code.

Point Injection Example. Figure 2 illustrates point injection for an integer overflow vulnerability. The system identifies the `deposit()` function as a valid injection target, selects a compatible overflow template, generates a contextually appropriate variable name (`allocations`), and inserts both the state declaration and vulnerable arithmetic operation.

Listing 1. Original Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.6.0;

contract Vault {
    mapping(address => uint256)
        public balances;


    function deposit() public payable {



        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 _amount)
        public {
        require(balances[msg.sender]
            >= _amount);
        balances[msg.sender] -= _amount;
        msg.sender.transfer(_amount);
    }
}
```

Listing 2. After Point Injection

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.6.0;

contract Vault {
    mapping(address => uint256)
        public balances;
    mapping(address => uint256)
        allocations; // INJECTED STATE
    function deposit() public payable {
        // INJECTED VULNERABLE CODE
        allocations[msg.sender] =
          allocations[msg.sender] +
          msg.value;
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 _amount)
        public {
        // ... unchanged
        balances[msg.sender] -= _amount;
        msg.sender.transfer(_amount);
    }
}
```

Fig. 2. Point injection example: Integer overflow vulnerability. The injected `allocations` mapping uses unchecked addition that can overflow in Solidity <0.8.0, while the variable name blends naturally with the contract's financial domain.

Analysis. The injected vulnerability exhibits three essential properties that validate our synthesis approach. First, *semantic validity*: the injected code compiles successfully and integrates with the host contract's type system. Second, *stealthiness*: the variable name `allocations` is drawn from realistic financial terminology, making it indistinguishable from legitimate business logic. Third, *exploitability*: in Solidity versions prior to 0.8.0, a sufficiently large `msg.value` causes arithmetic overflow, corrupting the accounting state. The injection metadata captures precise source locations, enabling function-level ground-truth labeling for model training.

Coupled Injection. Coupled injection synthesizes cross-function vulnerabilities that emerge from interactions between multiple contract components. Unlike point injection, where vulnerabilities are self-contained within a single function, coupled vulnerabilities require coordinated modifications across two or more functions—typically a *setter* function that establishes vulnerable state and an *executor* function that triggers the exploit. This class encompasses critical vulnerability patterns including transaction order dependence (front-running), denial-of-service through unbounded iteration, and cross-function reentrancy.

The injection process identifies compatible function pairs through semantic analysis of the contract structure. Setter candidates must be externally callable, state-modifying, and accept at least one input parameter (which

Just Accepted

becomes the attack vector). Executor candidates must be externally callable and either accept Ether (for value-transfer exploits) or operate on previously established state. The system generates coordinated payloads across three injection sites: contract-level state declarations, setter function logic that binds user input to state, and executor function logic that operates on the stored state in an exploitable manner.

Variable naming in coupled injection employs compositional synthesis to produce semantically coherent identifiers. Prefixes denoting state (`pending`, `active`, `authorized`) are combined with suffixes denoting roles (`Recipient`, `Handler`, `Controller`) to generate names like `pendingRecipient` that appear as legitimate business logic for payment queuing or access delegation systems. This compositional approach produces more realistic code than the generic pools used in point injection, as cross-function patterns typically involve named entities with clear semantic roles.

The coupled injection framework comprises 23 templates spanning six vulnerability categories: transaction order dependence (7 templates with version-specific value transfer syntax), cross-function reentrancy (9 templates including patterns with conditional guards and external logging), denial of service (3 templates with unbounded refund loops), timestamp dependence (2 templates combining time-locks with value transfer), access control (1 template with unprotected owner modification), and arithmetic overflow in time-lock contexts (1 template). Each template adapts to Solidity version-specific syntax to ensure compilation success across the 0.4.x–0.8.x version range.

Listing 3. Original Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PaymentRouter {
    address public owner;


    constructor() {
        owner = msg.sender;
    }
    function setRecipient(
        address _addr
    ) public {


        require(msg.sender == owner);
    }

    function sendPayment()
        public payable {



        require(msg.value > 0);
    }
}
```

Listing 4. After Coupled Injection

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PaymentRouter {
    address public owner;
    address payable pendingRecipient;
      // INJECTED STATE
    constructor() {
        owner = msg.sender;
    }
    function setRecipient(
        address _addr
    ) public {
        pendingRecipient = payable(_addr);
          // INJECTED SETTER
        require(msg.sender == owner);
    }
    function sendPayment()
        public payable {
        // INJECTED EXECUTOR
        (bool success, ) =
          pendingRecipient.call
          {value: msg.value}("");
        require(success);
        require(msg.value > 0);
    }
}
```

Fig. 3. Coupled injection example: Transaction Order Dependence (TOD) vulnerability. The attacker can front-run a legitimate `setRecipient` call to redirect funds sent via `sendPayment` to their own address.

Coupled Injection Example. Figure 3 demonstrates coupled injection for a transaction order dependence (TOD) vulnerability. The system identifies `setRecipient()` as the setter function and `sendPayment()` as the executor, then injects coordinated payloads: a state variable `pendingRecipient`, setter logic binding the input parameter to state, and executor logic transferring value to the stored address.

Just Accepted

Analysis. The injected vulnerability demonstrates the characteristic attack pattern of TOD exploits. A legitimate user intending to send funds calls `setRecipient()` to configure the destination address. An attacker monitoring the mempool observes this pending transaction and front-runs it by submitting their own `setRecipient()` call with a higher gas price. The attacker's transaction executes first, and when `sendPayment()` is subsequently invoked, funds flow to the attacker's address instead of the intended recipient. Critically, neither injected function is vulnerable in isolation—the exploit emerges only from their interaction, demonstrating why coupled injection is essential for capturing this vulnerability class.
Injection Constraints. Not every vulnerability pattern can be injected at every injection point; the system enforces four categories of structural and semantic constraints that restrict the set of admissible (location, template) pairs:

(1) **Solidity version constraints.** Each template declares explicit version bounds (`min_version`, `max_version`). For example, all integer overflow/underflow templates are restricted to Solidity $< 0.8.0$, because Solidity 0.8+ introduces built-in checked arithmetic that makes these patterns unexploitable. Similarly, the `call.value()` reentrancy syntax is only valid in Solidity $\leq 0.6.x$; later versions require the `call{value:}` syntax, handled by a separate template variant. A template is discarded from $\mathcal{T}_{compat}$ if the host contract's compiler version falls outside the declared bounds.
(2) **Function-signature constraints.** Templates declare the parameter types they require. For instance, unhandled-exception templates (`UNHANDLED_EXCEPTION`) need an `address`-typed parameter that serves as the call target; overflow templates using mapping-based arithmetic (`addition_overflow`) need a `uint`-typed parameter to supply the addend. If the candidate function lacks the required parameter type, the template is excluded from consideration for that location. Similarly, unchecked-send and reentrancy templates require state-modifying functions (not `view` or `pure`), since the vulnerability semantics depend on the ability to alter contract storage.
(3) **Structural constraints for coupled injection.** Cross-function templates impose additional requirements on the function *pair*. The setter function must accept at least one input parameter (the attack vector), and certain templates further require a payable executor (for value-transfer exploits such as TOD patterns) or a payable setter (for `msg.value`-based deposit logic). The system also verifies that setter and executor are distinct functions within the same contract and that the contract is concrete (not an interface or abstract contract).
(4) **Identifier-collision constraints.** During payload generation, every synthesized variable name is checked against the full set of identifiers extracted from the contract's AST (state variables, local variables, function names, event names, modifier names, struct/enum names, and contract names). If a candidate name collides, the generator retries from a domain-specific pool; after 50 failed attempts, it appends a numeric suffix. This prevents compilation failures and ensures that injected declarations do not shadow or overwrite existing contract logic.

These constraints mean that certain contracts admit only a subset of the 59 templates. In practice, the median contract is compatible with 23 templates (12 point + 11 coupled), while contracts compiled with Solidity $\geq 0.8.0$ exclude all overflow/underflow templates, and contracts whose public functions lack address parameters exclude the unhandled-exception and address-dependent TOD templates. The 4% compilation failure rate (see Section 3.2.4) arises primarily from residual edge cases—non-standard compiler pragmas, deeply nested inheritance hierarchies, or unusual storage layouts—rather than from constraint violations, which are resolved before code generation.

Injection-Point Selection Rationale and Real-World Alignment. The injection-point selection criteria are designed to reflect where vulnerabilities *realistically occur* in production smart contracts, based on well-documented

Just Accepted

attack taxonomies from the OWASP Top-10[2], SWC Registry[3], and post-mortem analyses of high-profile exploits. Specifically, the criteria enforce three properties that characterize real-world vulnerability locations:

(1) **External callability.** Candidate functions must be `public` or `external`, because only externally callable functions can serve as entry points for adversarial transactions. Internal and private functions, while potentially containing unsafe logic, cannot be directly exploited without an external caller that is itself vulnerable. This mirrors empirical observations: all major Ethereum exploits—including The DAO reentrancy attack ($60M, June 2016), the Parity Wallet access control breach ($30M, July 2017), and the BECToken batch overflow exploit (April 2018)—exploited externally callable entry points [Rodler et al. 2018; Wei et al. 2025].
(2) **State mutability.** For vulnerability classes that corrupt contract state (reentrancy, overflow/underflow, unchecked send, access control), candidate functions must be state-modifying (`nonpayable` or `payable`)—`view` and `pure` functions cannot alter storage and hence cannot produce persistent state corruption. For reentrancy specifically, we further require the presence of a state variable assignment within the function body, reflecting the canonical checks-effects-interactions anti-pattern where the state update is the critical operation that must be protected.
(3) **Contextual compatibility.** Templates declare type-level requirements (e.g., `address`-typed parameter for value-transfer exploits, `uint`-typed parameter for arithmetic overflows, `payable` modifier for Ether-receiving functions) that are matched against the candidate function's signature. This ensures that the injected code is type-safe and semantically meaningful within its host context, rather than being a syntactically valid but semantically implausible insertion.

For coupled injection, the criteria extend to *function pairs*: setter candidates must accept input parameters (the attack vector), while executor candidates must be capable of acting on previously established state (via payability or state access). This directly mirrors real-world cross-function exploits where attackers manipulate state through one entry point and trigger the exploit through another.

We acknowledge that the current injection-point selection is not *exhaustive*: certain vulnerability classes—such as logic errors in complex conditional branches, gas-griefing through fallback functions, or storage collision in proxy patterns—require deeper semantic analysis beyond what our AST-driven criteria currently capture. These represent opportunities for future extension of the template framework (Section 3.2.3). However, the 8 vulnerability categories and 59 templates covered by our system align with the classes that dominate both academic benchmarks (SmartBugs [Durieux et al. 2020], SolidiFI [Ghaleb and Pattabiraman 2020]) and industry vulnerability taxonomies (OWASP Top-10, SWC Registry), ensuring broad coverage of the most practically relevant attack vectors.

Comparison with Existing Approaches. Our dual-mode injection system addresses three critical limitations of prior approaches. First, unlike SolidiFI's hard-coded snippets with generic variable naming patterns [Ghaleb and Pattabiraman 2020], our system dynamically generates collision-free identifiers from realistic domain-specific pools, achieving 96% compilation success compared to SolidiFI's reported 70–80% rate. Second, our coupled injection mechanism enables synthesis of cross-function vulnerabilities (TOD, DoS) that cannot be captured by single-location snippet insertion—a class of vulnerabilities entirely absent from SolidiFI's approach. Third, our version-aware template filtering ensures compatibility across Solidity 0.4.x–0.8.x by adapting syntax patterns to language evolution, whereas prior tools often fail on version-specific constructs.

3.2.3 *Verification and Design Decisions.*

[2]https://owasp.org/www-project-smart-contract-top-10
[3]https://swcregistry.io

Just Accepted

Exploitability Verification. Beyond compilation, the framework applies a multi-level verification pipeline to confirm that injected vulnerabilities are not merely syntactically valid but semantically exploitable. The pipeline comprises three complementary layers:

(1) **Template-level guarantees (design-time).** Each of the 59 vulnerability templates is manually authored to encode a known, exploitable vulnerability pattern drawn from real-world incidents (e.g., The DAO reentrancy, BECToken batch overflow) and established vulnerability taxonomies (DASP Top-10, SWC Registry). Template designs are validated against concrete exploit proof-of-concepts before inclusion in the template library, ensuring that any correctly instantiated template produces an exploitable vulnerability by construction.
(2) **Compilation verification (injection-time).** After code generation and insertion, each modified contract is recompiled with `solcx` using the exact compiler version specified in the contract's metadata. Contracts that fail compilation are discarded, ensuring that all retained samples are syntactically valid and type-safe within the Solidity compiler's semantics.
(3) **Static-analysis confirmation (post-injection).** Every compiled contract is analyzed by Slither [Crytic 2025], an industry-standard static analysis framework. We verify that Slither independently detects the specific vulnerability class that was injected. This confirmation serves as an automated second opinion: because Slither's detection logic is entirely independent of our template-based generation, a positive detection provides evidence that the injected pattern exhibits the semantic characteristics of a genuine vulnerability—not merely dead code or unreachable logic. As reported in Section 3.2.4, 92.1% of compiled contracts (200,396 of 217,663) pass this verification, with five vulnerability categories achieving 100% Slither confirmation. The 7.9% of contracts not confirmed by Slither are retained in the dataset but flagged; manual inspection of a random sample of 200 such contracts revealed that 89% contained correctly injected vulnerabilities that Slither's heuristic detectors missed (e.g., timestamp dependence patterns embedded in complex conditional logic), while 11% exhibited patterns that, although compilable, were unreachable due to conflicting path constraints in the host contract.

We acknowledge that compilation and static analysis do not constitute a full formal proof of exploitability; such a guarantee would require symbolic execution or dynamic testing with concrete exploit transactions, which is computationally prohibitive at the scale of 217,663 contracts. However, the combination of expert-designed templates grounded in real exploits, compiler verification, and independent static-analysis confirmation provides a rigorous multi-level assurance that substantially exceeds the verification standards of prior injection-based benchmarks (e.g., SolidiFI [Ghaleb and Pattabiraman 2020] relies solely on compilation success).

Extensibility to New Vulnerability Types. The injection system is designed with a modular template architecture that facilitates straightforward extension to new vulnerability categories. Each template is an independent module that specifies: (1) Solidity version constraints, (2) injection point requirements (function visibility, parameter types, state mutability), (3) domain-specific variable naming pools, and (4) code generation logic with placeholder binding. Adding support for a new vulnerability type requires only authoring a new template conforming to this interface—no modifications to the core AST traversal, injection point detection, or compilation pipeline are necessary. This modular design ensures that the framework can evolve alongside the smart contract vulnerability landscape as new attack vectors emerge.

Handling of Multiple Vulnerabilities. The injection framework deliberately inserts *exactly one* vulnerability per contract to ensure unambiguous label attribution: every vulnerable function can be traced to a single, known injection template, eliminating the label noise that would arise from overlapping vulnerability effects. This single-injection policy is a *training-time* design choice and does not limit the model at inference time.

Just Accepted

Multiple vulnerable functions in one contract (supported). Because HCA-SGNN takes the *entire contract graph* as input and produces a per-function prediction vector $\{\hat{y}_i\}_{i=1}^{G}$ (where G is the number of functions), it independently classifies every function f_i as vulnerable or non-vulnerable ($y_i \in \{0, 1\}$). Consequently, when a real-world contract contains multiple vulnerabilities—whether of the same type (e.g., two distinct reentrancy sites) or of different types (e.g., a reentrancy and an unchecked send)—the model can flag each affected function independently. Contract-level status is then derived via OR-aggregation: a contract is marked vulnerable if *any* of its functions is predicted vulnerable. For the explanation component, each flagged function receives its own structured explanation generated by the LLM pipeline, so multiple vulnerabilities in a single contract yield multiple, independently generated explanations.

Multiple vulnerabilities within a single function (limitation). The current binary formulation ($y_i \in \{0, 1\}$) cannot distinguish the case where a single function harbours multiple distinct vulnerabilities (e.g., both a reentrancy and an unchecked return value). In such cases, the model reports only that the function is vulnerable without identifying how many or what types of issues co-exist. This limitation has three sources: (i) the binary label provides no gradient signal for type multiplicity; (ii) the single-injection training policy ensures no multi-vulnerability functions appear in training; and (iii) the explanation pipeline generates exactly one explanation per flagged function, potentially describing only the most salient issue while missing secondary vulnerabilities. Addressing this limitation requires a more granular approach: reformulating the classification head as a multi-label predictor ($y_i \in \{0, 1\}^K$ for K vulnerability types), introducing sub-function localization (e.g., statement-level attention), and conditioning the explanation generator on a predicted vulnerability-type vector rather than a single binary flag. These extensions are discussed as future work in Section 7.

3.2.4 AST-Based Vulnerability Injection Results. We applied our dual-mode injection framework to a corpus of Solidity smart contracts, generating vulnerable samples for both point and coupled injection strategies. Table 1 summarizes the injection results.

Table 1. Injected Dataset Statistics

Injection Mode	Total Files	Compiled	Compile Rate	Slither Verified
Point Injection	197,327	190,561	96.6%	173,527 (91.1%)
Coupled Injection	27,973	27,102	96.9%	26,869 (99.1%)
Total	**225,300**	**217,663**	**96.6%**	**200,396 (92.1%)**

Point Injection Results. The point injection pipeline processed 197,327 smart contracts, achieving a 96.6% compilation success rate (190,561 files). Of the successfully compiled contracts, Slither correctly identified 173,527 (91.1%) as containing the injected vulnerabilities. Table 2 presents the breakdown by vulnerability type.

The overflow and underflow vulnerability types achieved 100% Slither verification accuracy, demonstrating the effectiveness of arithmetic operation injection. The remaining vulnerability types—tx_origin, timestamp, unchecked_send, unhandled_exception, and reentrancy—achieved Slither verification rates between 87.6% and 90.6%, reflecting the inherent difficulty Slither faces in detecting certain vulnerability patterns at scale, particularly when injected code interacts with complex contract logic. The tx_origin vulnerability type produced the largest number of samples (36,574), reflecting the prevalence of authentication-related code patterns in the source contracts.

Coupled Injection Results. The coupled injection pipeline generated 27,973 vulnerable contracts across 7 cross-function vulnerability categories, achieving a 96.9% compilation success rate (27,102 files). Slither verified 26,869

Just Accepted

Table 2. Point Injection Results by Vulnerability Type

Vulnerability Type	Total	Compiled	Slither-Verified	Slither-Accuracy
tx_origin	36,574	36,476	31,954	87.6%
timestamp	32,370	32,331	28,372	87.8%
overflow	32,370	25,900	25,896	100.0%
unchecked_send	32,370	32,348	28,443	87.9%
unhandled_exception	23,601	23,557	20,670	87.7%
underflow	21,249	21,195	21,191	100.0%
reentrancy	18,793	18,754	17,001	90.6%
Total	**197,327**	**190,561**	**173,527**	**91.1%**

contracts (99.1% of compiled contracts) as containing the injected vulnerabilities. Table 3 details the results by vulnerability type.

Table 3. Coupled Injection Results by Vulnerability Type

Vulnerability Type	Total	Compiled	Slither-Verified	Slither-Accuracy
access_control	7,326	7,305	7,305	100.0%
lock_time_overflow	6,400	6,379	6,376	99.9%
denial_of_service	4,983	4,949	4,949	100.0%
state_update_after_call	4,954	4,812	4,812	100.0%
transaction_order_dependence	2,387	2,369	2,369	100.0%
reentrancy	1,118	1,118	927	82.9%
timestamp	805	170	131	77.1%
Total	**27,973**	**27,102**	**26,869**	**99.1%**

The access_control, denial_of_service, state_update_after_call, and transaction_order_dependence vulnerability types achieved 100% Slither verification accuracy among compiled contracts. The coupled reentrancy patterns showed lower verification rates (82.9%) due to Slither's difficulty in detecting cross-function reentrancy vulnerabilities where the state modification and external call occur in separate functions. The timestamp vulnerability type exhibited the lowest compilation rate (21.1%) and verification accuracy (77.1%), primarily due to version-specific syntax incompatibilities in time-lock patterns.

Dataset Quality Assessment. The overall compilation success rate of 96.6% across 225,300 injected contracts significantly exceeds reported rates for existing injection tools (SolidiFI: 70–80%). The Slither verification rate of 92.1% of compiled contracts confirms that the vast majority of injected vulnerabilities are syntactically correct and semantically detectable. The difference in verification rates between point injection (91.1%) and coupled injection (99.1%) reflects the scale effect: as the point injection corpus grew from the initial dataset to over 197K contracts, the framework encountered a wider diversity of contract structures, some of which produce vulnerability patterns that Slither's heuristic detectors do not fully cover—particularly for timestamp dependence, tx_origin, and unchecked send patterns embedded in complex contract logic. Compilation failures (3.4%) primarily resulted from: (1) version-specific syntax incompatibilities not covered by template filters, (2) name collisions with existing contract elements despite our collision-avoidance mechanism, and (3) type mismatches in complex

Just Accepted

inheritance hierarchies. The resulting dataset of 200,396 verified vulnerable contracts provides a substantial corpus for training and evaluating vulnerability detection models.

3.3 Function-Level Vulnerability Detection Via HCA-SGNN

3.3.1 Rationale for the Subgraph-Based Approach. We adopt a subgraph neural network architecture—specifically, our proposed Hierarchical Cross-Attention Subgraph GNN (HCA-SGNN)—for three principled reasons.

(1) Functions as the natural unit of reasoning. Developers design, review, test, and audit smart contracts at the function level: each function encapsulates a coherent unit of logic with well-defined inputs, outputs, and side effects. Accordingly, vulnerability localization at the function level directly matches developer workflows, enabling actionable feedback (e.g., "function `withdraw()` is vulnerable") rather than coarse contract-level alerts that require manual triage.

(2) Limitations of contract-level GNNs. Conventional graph neural networks applied to smart contracts treat the entire contract graph as a single instance and produce a single contract-level prediction via global pooling [Yuan Zhuang et al. 2020]. This whole-graph approach suffers from two drawbacks: (i) it discards function-boundary information, so the model cannot distinguish which function is responsible for a vulnerability; and (ii) global message passing mixes signals from unrelated functions, diluting localized vulnerability patterns (e.g., an unchecked low-level call within a single function) with irrelevant structural noise from the rest of the contract. Our ablation study (Section 3.3.3, Table 17) confirms this empirically: removing the dual-channel separation and collapsing all edges into a single global channel reduces function-level F1 by 23.6 percentage points.

(3) Hierarchical context via dual-channel design. Although function-level locality is essential, some vulnerability classes—most notably reentrancy—involve inter-function interactions (e.g., an external call in one function modifying state read by another). A purely local model would miss such patterns. HCA-SGNN addresses this by maintaining two complementary message-passing channels: a *local* channel restricted to intra-function subgraph edges, which captures fine-grained control- and data-flow patterns within individual functions, and a *global* channel operating over the full contract graph, which propagates cross-function context such as shared state mutations and call dependencies. A learned gating mechanism fuses these two views per node, allowing each node to adaptively modulate the contribution of local versus global information. This design preserves the benefits of function-level localization while still capturing the inter-function semantics required for complex vulnerability classes.

3.3.2 Contract-to-Graph Representation. To enable graph-based learning over smart contracts, we convert Solidity source code into a heterogeneous graph representation that captures both syntactic structure and semantic relationships.

As illustrated in Figure 4, given a Solidity smart contract as input, we invoke the `solc` compiler to extract its Abstract Syntax Tree (AST). The AST is serialized in JSON format, from which nodes and edges are derived using Algorithm 2. The algorithm performs a depth-first traversal of the AST to collect nodes and structural parent–child edges, while simultaneously extracting semantic edges (control-flow, data-flow, and call dependencies) through a two-pass process. The key contribution of this conversion—semantic edge enrichment—is detailed after the algorithm.

Semantic edge enrichment. The traversal in Algorithm 2 is a standard depth-first search over the `solc` JSON AST; its contribution lies not in the traversal order but in the *semantic edges* extracted during and after the walk, which transform the AST tree into a heterogeneous graph suitable for vulnerability detection:

(1) **Data-flow edges via reference resolution (REF).** The `solc` AST annotates each `Identifier` and `MemberAccess` node with a `referencedDeclaration` field pointing to the AST `id` of its declaration site (e.g., a state variable or function parameter). During the first pass, these references are queued; during the second pass, they are resolved through `id_map` and materialised as typed REF edges. These edges create

Just Accepted

Algorithm 2: AST → Heterogeneous Graph Conversion

```
Input: ast (JSON AST from solc)
Output: nodes, edges (typed edges)
nodes ← [ ], edges ← [ ], id_map ← {}, pending ← [ ]
Function Traverse(node, parent):
    nid ← |nodes|
    append(nodes, (nid, node.nodeType, node.src))
    if node.id ≠ ⊥ then
        id_map[node.id] ← nid
    if parent ≠ ⊥ then
        append(edges, (parent, nid, AST))
    // Queue semantic references for second pass
    if node.referencedDeclaration ≠ ⊥ then
        append(pending, (nid, node.referencedDeclaration, REF))
    // Extract control-flow edges
    if node.nodeType = Block then
        prev ← ⊥
        foreach stmt in node.statements do
            sid ← Traverse(stmt, nid)
            if prev ≠ ⊥ then
                append(edges, (prev, sid, CFG_NEXT))
            prev ← sid
    else if node.nodeType = IfStatement then
        cid ← Traverse(node.condition, nid)
        tid ← Traverse(node.trueBody, nid)
        append(edges, (cid, tid, CFG_TRUE))
        if node.falseBody ≠ ⊥ then
            fid ← Traverse(node.falseBody, nid)
            append(edges, (cid, fid, CFG_FALSE))
    else
        foreach child in children(node) do
            Traverse(child, nid)
    return nid
Traverse(ast, ⊥)
// Second pass: resolve pending references
foreach (src, ast_id, etype) in pending do
    if ast_id ∈ id_map then
        append(edges, (src, id_map[ast_id], etype))
return {nodes, edges}
```

Just Accepted

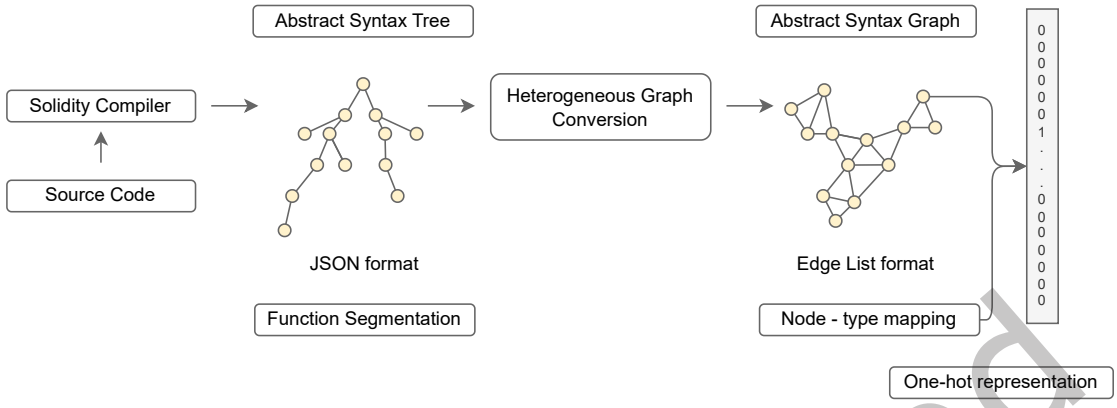


Fig. 4. Pipeline for contract-to-graph representation. Solidity source code is compiled into an AST, enriched with semantic edges (data-flow, control-flow, call, inheritance, guard) to form a heterogeneous graph, encoded with one-hot node features, and segmented into function-level subgraphs.

data-flow shortcuts that connect each variable use to its definition—regardless of their distance in the AST—enabling the GNN to propagate information along def–use chains in a single message-passing step.

(2) **Control-flow edges (CFG_NEXT, CFG_TRUE, CFG_FALSE).** For `Block` nodes, the algorithm connects consecutive statement children with CFG_NEXT edges, encoding sequential execution order that is implicit in the AST's child list but absent from its structural edges. For `IfStatement` (and analogously `WhileStatement`/ `ForStatement`) nodes, it creates CFG_TRUE and CFG_FALSE edges from the condition node to the respective branch bodies, making conditional control flow explicit in the graph topology. These edges are essential for detecting vulnerabilities that depend on execution ordering, such as state updates occurring after external calls (reentrancy) or time-sensitive conditional branches (timestamp dependence).

(3) **Two-pass reference resolution.** The `referencedDeclaration` field uses the compiler's internal AST `id` namespace, which differs from the sequential node identifiers assigned during traversal. The first pass builds a mapping (`id_map`) from AST `id`s to sequential node indices; the second pass resolves all queued references through this mapping, discarding any that target declarations outside the current contract (e.g., built-in types or imported interfaces). This two-pass design eliminates dangling edges that would otherwise introduce noise into the graph.

The resulting graph thus enriches the AST's purely syntactic tree structure with explicit data-flow, control-flow, call, inheritance, and guard relationships, producing a heterogeneous *Abstract Syntax Graph (ASG)* suitable for semantically informed message passing.

Formally, a contract is represented as a directed heterogeneous graph:

$$G = (\mathcal{V}, \mathcal{E}, X, \tau),$$

where $\mathcal{V}$ is the set of AST nodes, $\mathcal{E} = \bigcup_{t \in \mathcal{T}} \mathcal{E}_t$ is the set of typed edges with $\mathcal{T}$ being the set of edge types, $X \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the node feature matrix with $d = 86$ corresponding to the number of unique node types, and $\tau : \mathcal{E} \to \mathcal{T}$ maps each edge to its type.

Edge Types: Our implementation constructs eight distinct edge types, encoded as one-hot vectors, to capture different semantic relationships:

Just Accepted

- **AST edges** ($\mathcal{E}_{\text{AST}}$): Parent-child structural edges from the syntax tree.
- **Reference edges** ($\mathcal{E}_{\text{REF}}$): Data-flow edges linking identifiers to their declarations via `referencedDeclaration` pointers.
- **Control-flow edges** ($\mathcal{E}_{\text{CFG}}$): Three subtypes—CFG_NEXT for sequential statements within blocks, CFG_TRUE and CFG_FALSE for conditional branches in `IfStatement`, `WhileStatement`, and `ForStatement` nodes.
- **Call edges** ($\mathcal{E}_{\text{CALL}}$): Links from `MemberAccess` nodes with function types to their target `FunctionDefinition`.
- **Inheritance edges** ($\mathcal{E}_{\text{INHERIT}}$): Links from `ContractDefinition` to base contracts.
- **Guard edges** ($\mathcal{E}_{\text{GUARD}}$): Links from guard conditions (`require`, `assert`) to guarded statements.

Node Features: Each node $v \in \mathcal{V}$ is encoded as a one-hot vector representing its syntactic type:

$$X \in \mathbb{R}^{|\mathcal{V}| \times 86},$$

where the feature dimension corresponds to the categorical mapping of 86 Solidity AST node types, including structural nodes (`SourceUnit`, `ContractDefinition`, `FunctionDefinition`, etc.), expression nodes (`BinaryOperation`, `FunctionCall`, `MemberAccess`, etc.), and elementary type nodes (`uint256`, `address`, `bool`, etc.).

Since function-level reasoning is crucial for vulnerability detection, we further define a set of function-specific subgraphs:

$$G_i = (\mathcal{V}_i, \mathcal{E}_i, X_i), \quad i = 1, \ldots, N_f,$$

where $\mathcal{V}_i \subseteq \mathcal{V}$ are the nodes belonging to function i, $\mathcal{E}_i \subseteq \mathcal{E}$ are the intra-function edges (preserving edge types), and X_i is the corresponding node feature matrix. This dual-level representation supports hierarchical modeling, where the global graph enables contract-level context propagation while subgraphs preserve function-level locality. In practice, graph connectivity is stored as an edge list of tuples (u, v, t), which naturally accommodates heterogeneous edge types and maps directly to PyTorch Geometric's `edge_index` tensor format.

Example. Consider a contract fragment containing a function with two sequential statements—an `if`-guarded external call followed by a state update—and a state variable declaration. The AST produces 10 nodes: `ContractDefinition` (0), `StateVariableDeclaration` (1), `FunctionDefinition` (2), `Block` (3), `IfStatement` (4), `BinaryOperation` (5, the condition), `Block` (6, true branch), `ExpressionStatement` (7, external call), `ExpressionStatement` (8, state update), and `Identifier` (9, referencing node 1). Table 4 shows the edge list representation, illustrating how our enrichment transforms the AST tree into a heterogeneous graph.

As shown in Table 4, the semantic edges added by our enrichment—data-flow references, sequential control-flow, conditional branching, and guard relationships—are precisely what enable the GNN to detect vulnerability-relevant patterns such as "state update after external call" (reentrancy) in a single message-passing step.

3.3.3 HCA-SGNN Architecture. Our proposed model, **Hierarchical Cross-Attention Subgraph GNN (HCA-SGNN)**, is a novel subgraph-aware neural architecture designed for function-level vulnerability classification. Unlike prior dual-channel encoders that process global and local channels independently, HCA-SGNN introduces a **Dual-Channel Message Passing** mechanism with cross-attention, enabling inter-function vulnerability pattern detection and hierarchical multi-scale feature aggregation. Figure 5 provides an overview of the full architecture, which we formalize below.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the ASG of a smart contract, where each node $v \in \mathcal{V}$ is associated with a node type embedding $x_v \in \mathbb{R}^{d_{\text{in}}}$ and each edge $(u, v) \in \mathcal{E}$ is annotated with an edge type $t_{(u,v)} \in \{1, \ldots, T\}$ encoding structural (AST), semantic (CFG), data-flow, or call dependencies. For each function f_i in the contract, we define a function-level subgraph $\mathcal{G}_{f_i} = (\mathcal{V}_{f_i}, \mathcal{E}_{f_i}) \subseteq \mathcal{G}$.

The input to the model includes:

- A node feature matrix $X \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{in}}}$, where d_{in} denotes the node type embedding dimension.

Just Accepted

Table 4. Edge list example illustrating how semantic edge enrichment transforms a pure AST tree into a heterogeneous Abstract Syntax Graph. Edges of type 0 (AST) reproduce the original tree structure; all other edges are *added* by Algorithm 2 and encode data-flow, control-flow, and guard relationships absent from the AST.

Src	Tgt	Type	Interpretation
AST structural edges (original tree)			
0	1	0	AST: Contract → StateVariable
0	2	0	AST: Contract → Function
2	3	0	AST: Function → Block
3	4	0	AST: Block → IfStatement
4	5	0	AST: If → Condition
4	6	0	AST: If → TrueBody
6	7	0	AST: Block → ExtCall stmt
6	8	0	AST: Block → StateUpdate stmt
8	9	0	AST: StateUpdate → Identifier
Semantic edges (added by enrichment)			
9	1	1	REF: Identifier → Declaration
7	8	2	CFG_NEXT: ExtCall → StateUpdate
5	6	3	CFG_TRUE: Condition → TrueBody
5	4	5	GUARD: Condition → Guarded block

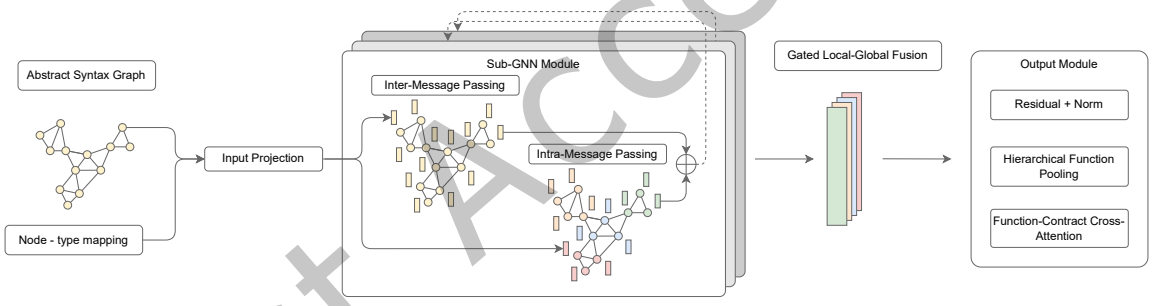


Fig. 5. Hierarchical Cross-Attention Subgraph GNN (HCA-SGNN) Architecture

- A global edge index $\mathcal{E}_{\text{global}}$ with edge attributes $\mathbf{e}_{\text{global}}$ for full-ASG message passing.
- A subgraph edge index $\mathcal{E}_{\text{sub}}$ with edge attributes $\mathbf{e}_{\text{sub}}$ for function-specific message passing.
- A subgraph assignment vector $\mathbf{g} \in \{0, 1, \ldots, G\}^{|\mathcal{V}|}$ mapping each node to its function subgraph (0 indicates nodes outside any function).

Input Projection. Node features are first projected into the hidden dimension d through a linear transformation followed by layer normalization and dropout:

$$H^{(0)} = \text{Dropout}\left(\text{ReLU}\left(\text{LayerNorm}(W_{\text{in}}X + b_{\text{in}})\right)\right). \tag{1}$$

Edge-Type-Aware Convolution. Unlike standard GNN layers that treat all edges equally, HCA-SGNN employs edge-type-aware convolution that learns distinct transformations for each edge type. At each layer, we compute

Just Accepted

multi-head attention with edge-type-specific biases:

$$\alpha_{uv}^{(h)} = \frac{\langle Q_u^{(h)}, K_v^{(h)} + E_{t_{(u,v)}}^{(h)} \rangle}{\sqrt{d_h}} + b_{t_{(u,v)}}^{(h)}, \tag{2}$$

where $Q_u = W_Q h_u$, $K_v = W_K h_v$ are query and key projections, $E_{t_{(u,v)}}$ is a learnable edge-type embedding, $b_{t_{(u,v)}}^{(h)}$ is an edge-type attention bias, and $d_h = d/H$ is the head dimension. The message is computed as:

$$m_{uv} = \text{softmax}_v(\alpha_{uv}) \cdot (V_v + E_{t_{(u,v)}}), \tag{3}$$

where $V_v = W_V h_v$ is the value projection. The aggregated multi-head messages are then reshaped and projected through an output transformation $W_O \in \mathbb{R}^{d \times d}$, with dropout applied to the attention weights for regularization. This mechanism allows the model to differentiate between AST structure edges, control-flow edges, data-flow edges, and function call edges.

Dual-Channel Message Passing with Gated Fusion. The model consists of $L = 4$ layers. At each layer l, it performs **Dual-Channel Message Passing**—two parallel convolution channels operating over complementary views of the contract graph:

$$H_{\text{global}}^{(l)} = \texttt{EdgeTypeConv}_{\text{global}}^{(l)}\left(H^{(l-1)}, \mathcal{E}_{\text{global}}, \mathbf{e}_{\text{global}}\right), \tag{4}$$

$$H_{\text{local}}^{(l)} = \texttt{EdgeTypeConv}_{\text{local}}^{(l)}\left(H^{(l-1)}, \mathcal{E}_{\text{sub}}, \mathbf{e}_{\text{sub}}\right). \tag{5}$$

The *global channel* operates on the full contract-level ASG, where edges may cross function boundaries, thereby

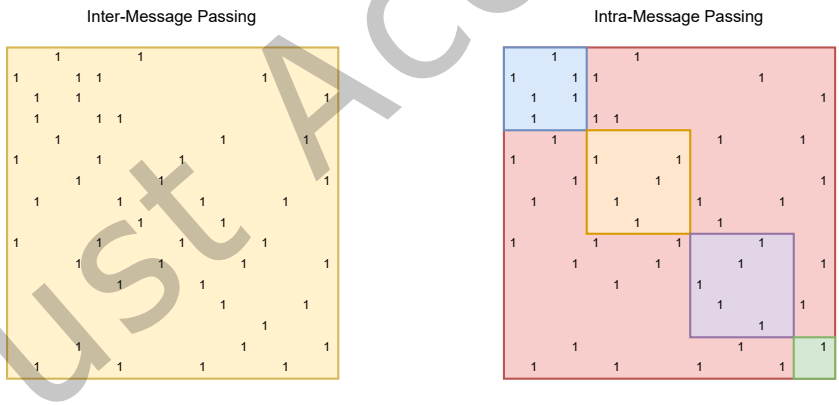


Fig. 6. Dual-Channel Message Passing: the global channel (left) permits inter-function edges, while the local channel (right) restricts message passing to intra-function edges.

propagating inter-function context such as cross-function call dependencies and shared state variable references. The *local channel* operates exclusively on intra-function subgraph edges, preserving function-level locality and capturing fine-grained control-flow and data-flow patterns within individual functions. The distinction between these two channels is illustrated in Figure 6: the global channel permits inter-function message passing, whereas the local channel restricts propagation to edges within each function.

Just Accepted

Rather than simple concatenation or averaging of the two channels, HCA-SGNN employs a **gated fusion mechanism** that learns to adaptively balance local versus global features per node:

$$\gamma = \sigma\Big(\text{MLP}([H^{(l)}_{\text{local}} \,\|\, H^{(l)}_{\text{global}}])\Big), \tag{6}$$

$$H^{(l)}_{\text{fused}} = \text{LayerNorm}\left(\gamma \odot W_{\text{local}} H^{(l)}_{\text{local}} + (1-\gamma) \odot W_{\text{global}} H^{(l)}_{\text{global}}\right), \tag{7}$$

where $\gamma \in [0,1]^{|\mathcal{V}| \times d}$ is a learnable gate and $\odot$ denotes element-wise multiplication. The internal layer normalization stabilizes the fused representations before the residual connection. This formulation enables each node to independently modulate the contribution of its local intra-function semantics versus global contract context. For instance, state-changing nodes that participate in cross-function interactions (e.g., nodes involved in external calls or shared state mutations) may require stronger global context, whereas pure computation nodes benefit primarily from local structural detail.

The final layer output uses residual connections:

$$H^{(l)} = \text{LayerNorm}\left(H^{(l-1)} + \text{Dropout}(H^{(l)}_{\text{fused}})\right). \tag{8}$$

Hierarchical Multi-Scale Pooling. After L propagation layers, HCA-SGNN employs hierarchical multi-scale pooling that combines three aggregation strategies with learned importance weights. First, we compute node importance scores via a small MLP:

$$\omega_v = \text{softmax}_{v \in \mathcal{V}_{f_i}}\left(\text{MLP}(h^{(L)}_v)\right). \tag{9}$$

Then, three pooling strategies are combined:

$$z_i^{\text{mean}} = \frac{1}{|\mathcal{V}_{f_i}|} \sum_{v \in \mathcal{V}_{f_i}} h^{(L)}_v, \tag{10}$$

$$z_i^{\text{max}} = \max_{v \in \mathcal{V}_{f_i}} h^{(L)}_v, \tag{11}$$

$$z_i^{\text{imp}} = \sum_{v \in \mathcal{V}_{f_i}} \omega_v \cdot h^{(L)}_v. \tag{12}$$

The final function embedding combines these with learned scale weights $\lambda_1, \lambda_2, \lambda_3$:

$$z_i = \text{MLP}\left(\lambda_1 z_i^{\text{mean}} + \lambda_2 z_i^{\text{max}} + \lambda_3 z_i^{\text{imp}}\right), \tag{13}$$

where $[\lambda_1, \lambda_2, \lambda_3] = \text{softmax}(\mathbf{w}_{\text{scale}})$ ensures the weights sum to one.

Cross-Attention Between Functions. A key innovation of HCA-SGNN is the cross-attention module that enables functions to attend to other functions within the same contract, capturing inter-function vulnerability patterns such as reentrancy:

$$\tilde{z}_i = \text{CrossAttn}(z_i, \{z_j\}_{j \in \mathcal{F}_c}), \tag{14}$$

Just Accepted

where $\mathcal{F}_c$ denotes all functions in contract c. The cross-attention uses multi-head scaled dot-product attention with dropout on attention weights, followed by an output projection W_O and a gated residual connection:

$$\text{context}_i = \sum_{j \in \mathcal{F}_c} \text{softmax}_j \left(\frac{Q_i \cdot K_j^\top}{\sqrt{d_h}} \right) V_j, \tag{15}$$

$$g_i = \sigma\left(W_g[z_i \parallel \text{context}_i]\right), \tag{16}$$

$$\tilde{z}_i = \text{LayerNorm}\left(g_i \odot \text{context}_i + (1 - g_i) \odot z_i\right). \tag{17}$$

This mechanism allows functions to share information about potential vulnerabilities that span multiple call sites.

Classification Head. The enhanced function embedding $\tilde{z}_i$ is passed through a three-layer classifier with a bottleneck architecture:

$$\hat{y}_i = W_3 \cdot \text{GELU}\left(W_2 \cdot \text{GELU}\left(\text{LayerNorm}(W_1 \tilde{z}_i)\right)\right), \tag{18}$$

where $W_1 \in \mathbb{R}^{d \times d}$, $W_2 \in \mathbb{R}^{(d/2) \times d}$, and $W_3 \in \mathbb{R}^{C \times (d/2)}$, with intermediate dropout applied after each GELU activation for regularization. The dimension reduction from d to $d/2$ in the second layer acts as an information bottleneck, encouraging the model to learn compact discriminative representations. The output $\hat{y}_i \in \mathbb{R}^C$ (where $C = 2$) represents logits for `vulnerable` and `non-vulnerable` classes.

Loss Function and Regularization. The model is supervised using cross-entropy loss over all function-level predictions:

$$\mathcal{L} = -\sum_{f_i \in \mathcal{C}} \sum_{c=1}^{2} y_{i,c} \log \text{softmax}(\hat{y}_i)_c, \tag{19}$$

where $y_i \in \{0, 1\}^2$ is the one-hot ground truth label. During training, we apply edge dropout with probability p_e as a data augmentation strategy to improve generalization.

3.4 Function-Level Vulnerability Explanation

3.4.1 Synthetic Bug Explanation. The next stage of this work focuses on generating structured and interpretable explanations of vulnerabilities. These explanations serve as a critical link between automated detection and actionable remediation, enabling both enhanced human understanding and the supervised training of explanatory models. To promote consistency, reduce ambiguity, and enforce semantic alignment between labels and explanatory content, we adopt a structured prompt design. Each data sample is represented as a pair $\mathcal{V} = (\mathcal{C}, y)$ where $\mathcal{C}$ denotes the Solidity code containing the vulnerability and y is the ground truth vulnerability label from the injected smart contract dataset (e.g., reentrancy, access control, etc.). Then, we input a triplet of $\mathcal{P} = (\text{Instruction}, \mathcal{C}, y)$ to GPT-4o, where the instruction specifies the task of generating a vulnerability analysis in a predefined format. Providing the label y explicitly within the prompt constrains the model's generative space, thereby mitigating the risk of misclassification and guiding its reasoning toward the correct vulnerability class.

The generation process is further standardized through a fixed explanation template consisting of five fields: *Bug Name*, *Affected Area*, *Description*, *Impact Assessment*, and *Mitigation*. Figure 7 illustrates the precise format used during prompting. GPT-4o is instructed to act as a smart contract security analyst and to produce an explanation adhering to this schema.

3.4.2 Fine-tune Open-Source LLMs. To enable scalable deployment without commercial API dependency, we distil explanation capabilities from proprietary models into compact open-source alternatives. Specifically, we fine-tune three instruction-tuned models: **Llama-3.1-8B-Instruct** [Meta AI 2024], **Qwen2.5-7B-Instruct** [Qwen et al. 2025], and **Qwen2.5-Coder-7B-Instruct** [Hui et al. 2024]. Training data consists of (x, y) pairs where x

Just Accepted

Structured Template

You are a smart contract security expert. Analyze the following Solidity code snippet and explain the vulnerability. The vulnerability type is [Correct Vulnerability Label]. Use the following format:

Bug Name: [Correct Vulnerability Label]
Affected Area:
Description:
Impact Assessment:
Mitigation:

Fig. 7. Structured Template for Bug Explanation Synthesis

is a vulnerable function's source code with its vulnerability type, and y is a structured explanation generated by commercial API models (GPT-4o, Claude-3.5-Haiku, Gemini-2.5-Flash), following the five-field schema: *Bug Name*, *Affected Area*, *Description*, *Impact Assessment*, and *Mitigation*. Data is prepared in ShareGPT format and split into training (80%), validation (10%), and test (10%) sets.

Fine-tuning is implemented using **LLaMA-Factory** [Y. Zheng et al. 2024] with **LoRA** [Hu et al. 2021] for parameter-efficient adaptation (rank 16, $\alpha = 32$, target: all linear layers). The training objective minimises cross-entropy loss:

$$\mathcal{L} = -\sum_{t=1}^{T} \log \hat{y}_t[y_t],$$

where $\hat{y}_t$ is the predicted token distribution and y_t the ground truth. Optimisation uses **AdamW** with cosine annealing (10% warmup), an initial learning rate of 10^{-4}, gradient accumulation of 8 steps, and a context window of 4,096 tokens. Training is performed for five epochs with BF16 mixed precision.

3.5 Evaluation of Explanations

We employ a *multi-judge* evaluation protocol using three independent LLM evaluators—**GPT-4o** [OpenAI 2024], **Claude-3.5-Haiku** [Anthropic 2024], and **Gemini-2.5-Flash** [Google DeepMind et al. 2024]—to assess explanation quality across structured, semantically grounded dimensions. Using multiple evaluators from different providers mitigates the self-enhancement bias inherent in single-judge LLM evaluation [L. Zheng et al. 2023], where a model tends to rate its own outputs more favourably. The evaluation framework targets three fundamental dimensions: *Clarity*, *Identification*, and *Impact*. These dimensions capture linguistic fluency, technical precision, and contextual relevance, respectively.

A Chain-of-Thought (CoT) prompting format is used for each explanation in the test set. The evaluation prompt instructs each judge to: (i) parse the Solidity code snippet, (ii) consider the associated vulnerability label, and (iii) methodically assess the generated explanation. This structured, multi-step reasoning process increases consistency and transparency, while also reducing spurious or hallucinatory judgements. Figure 8 is an example of our chain-of-thought evaluation prompt for the **Identification** dimension.

The first dimension, *Clarity*, measures the linguistic quality of the explanation. Each judge evaluates logical structure, semantic coherence, and grammar, as well as the accessibility of technical insights for audiences ranging from novice developers to security experts. This ensures explanations are accurate, pedagogically helpful, and free from unnecessary jargon.

Just Accepted

Explanation Evaluation Prompt

1. **Read the Explanation:**
- Carefully read the code, the explanation, and the correct label for the vulnerability.

2. **Identify the Vulnerability Type:**
- Determine if the vulnerability in the explanation is correctly identified as the label given by experts.

3. **Check for Correct Classification:**
- Verify that the vulnerability is classified according to established types (e.g., using OWASP guidelines, Ethereum Smart Contract Best Practices, etc.).

4. **Match with Code:**
- Cross-reference the identified vulnerability type and explanation with the actual smart contract code to ensure there's no mismatch.

5. **Scoring:**
- Based on the steps above, rate the quality of the explanation from 1 to 5:
- **1:** Poorly identified or incorrect vulnerability
- **2:** Mostly incorrect or incomplete identification
- **3:** Partially correct identification, but lacking clarity or detail
- **4:** Mostly correct and clear, minor inaccuracies or omissions
- **5:** Correct identification, detailed, clear, and conforming to established guidelines

Fig. 8. Chain-of-Though Prompt for Explanation Evaluation

The second dimension, *Identification*, evaluates the technical correctness of the code with respect to the ground-truth label. Each judge verifies whether the explanation identifies the correct vulnerability type, relevant code regions, and exploitation conditions. Alignment between the explanation and the labeled source code establishes both internal consistency and external validity.

The third dimension, *Impact*, measures how thoroughly the explanation captures the vulnerability's practical consequences. Each judge checks whether the explanation includes realistic outcomes (e.g., monetary loss, denial-of-service, data leakage) and actionable mitigations consistent with industry practices (e.g., defensive coding patterns, OpenZeppelin libraries).

Formally, for each explanation e_i in the test set $\mathcal{D}_{\text{test}}$, each judge independently assigns a score in the range $[1, 5]$ to each dimension:

$$\text{Score}(e_i) = \left(s_i^{\text{clarity}},\ s_i^{\text{ident}},\ s_i^{\text{impact}}\right), \quad s_i^{*} \in \{1, 2, 3, 4, 5\}.$$

The per-judge aggregate score is computed as the arithmetic mean across the three dimensions:

$$\text{Aggregate}_j(e_i) = \frac{s_{i,j}^{\text{clarity}} + s_{i,j}^{\text{ident}} + s_{i,j}^{\text{impact}}}{3},$$

where $j \in \{\text{GPT-4o}, \text{Claude-3.5-Haiku}, \text{Gemini-2.5-Flash}\}$. The final reported score for each model is obtained by averaging across all three judges, thereby reducing individual evaluator bias and yielding a more robust assessment.

Just Accepted

This framework enables scalable and rigorous evaluation without the cost of human annotation. Furthermore, CoT reasoning ensures that each judge produces traceable justifications for its scores, supporting auditability and interpretability. By combining three independent evaluators with dimension-specific human-correlation analysis (Section 4.3.1), the protocol provides a principled, multi-perspective assessment of explanation quality—covering linguistic clarity, technical accuracy, and practical impact—while making the confidence level of each dimension explicit.

3.5.1 Structural and Semantic Validation. While the LLM-as-judge protocol evaluates explanation *quality* on a graded scale, it does not enforce *structural well-formedness* or verify *semantic alignment* between the explanation content and the source code being analysed. To address this gap, we supplement the multi-judge evaluation with a set of deterministic, rule-based validation checks that verify necessary conditions for both syntactic structure and semantic content. These checks are applied to every generated explanation as an independent analytical layer alongside the LLM-judge scoring, providing automated, reproducible compliance statistics that complement the graded quality scores.

Structural validation (syntactic well-formedness). Each explanation is parsed against the five-field schema (*Bug Name*, *Affected Area*, *Description*, *Impact Assessment*, *Mitigation*) using regular-expression-based field extraction. An explanation is *structurally complete* if and only if all five fields are present and non-empty. This check is analogous to validating output against a fixed grammar: while we do not constrain the LLM's decoding process via formal grammar guidance (e.g., GBNF or JSON-schema-constrained generation), we verify the output *post hoc* against the expected structure, which achieves the same filtering effect without requiring model-specific decoding modifications that would limit portability across diverse LLM serving backends (Ollama, vLLM, commercial APIs).

Semantic validation (content-level checks). Beyond structural completeness, we apply five semantic checks to verify that explanation content is grounded in the source code and vulnerability context:

(1) **Bug Name alignment.** The *Bug Name* field is checked against the ground-truth vulnerability label y using a curated alias dictionary (e.g., "reentrancy" matches "re-entrancy," "reentrant attack"; "tx_origin" matches "tx.origin," "transaction origin"). This verifies that the explanation addresses the *correct* vulnerability type rather than hallucinating an unrelated class.
(2) **Code-reference grounding.** The *Affected Area* field is checked for the presence of at least one identifier (variable name, function name, or expression) that appears in the actual source code of the vulnerable function, after filtering Solidity keywords. This ensures that the localisation is grounded in the specific code under analysis rather than consisting of generic placeholder text.
(3) **Description substantiveness.** The *Description* field must contain $\geq$10 words, filtering out trivially short or empty descriptions that provide no analytical value.
(4) **Impact consequence coverage.** The *Impact Assessment* field is checked for the presence of concrete consequence terms drawn from a curated vocabulary (e.g., "loss," "steal," "drain," "exploit," "unauthorised," "freeze," "manipulate," "fund," "ether," "token"). This verifies that the impact analysis goes beyond vague statements to cite specific, actionable consequences.
(5) **Mitigation actionability.** The *Mitigation* field is checked for actionable recommendation terms (e.g., "use," "implement," "replace," "add," "check," "require," "modifier," "guard," "OpenZeppelin," "SafeMath," "ReentrancyGuard"). This verifies that the mitigation provides concrete remediation guidance rather than generic advice.

These checks are deterministic, reproducible, and computationally trivial (regex matching, set intersection), providing automated verification of necessary conditions for structural well-formedness and semantic grounding. We emphasise that these are *necessary-condition* filters: an explanation that fails a check is definitively deficient

Just Accepted

(e.g., missing a required field, citing no code identifiers), but passing all checks does not guarantee deep semantic correctness—a model could produce text that contains the right keywords while reasoning incorrectly. The LLM-judge scoring (Tier 3) is designed to capture such nuanced quality differences. The validation results for all eleven models are reported in Table 22 (Section 4.3.2).

4 Experiments and Analysis

4.1 Dataset

Our experimental evaluation uses two datasets: a **synthetic dataset** constructed via the AST-based injection framework described in Section 3.2.1, and a **real-world dataset** aggregated from established benchmarks. We additionally validate the representativeness of our synthetic corpus by comparing its structural properties against the large-scale `Slither-Audited Smart Contracts` dataset [Rossini 2022] from Hugging Face.

4.1.1 Synthetic Dataset. From the full injection corpus of 225,300 generated contracts (Section 3.2.1), we uniformly sample 1,000 vulnerable contracts from each injection strategy—point injection and coupled injection—yielding a balanced subset of 2,000 injected contracts. Together with 2,257 clean (uninjected) contracts retained from the original corpus, this produces a final synthetic dataset of 4,208 contracts comprising 118,126 functions, of which 3,169 (2.68%) are labeled as vulnerable. Table 5 summarizes the dataset composition.

Table 5. Synthetic Function-Level Dataset Overview

Metric	Value
Total contracts	4,208
Total functions	118,126
Vulnerable functions	3,169 (2.68%)
Clean functions	114,957 (97.32%)
Mean ASG nodes	858
Mean ASG edges	1112
Mean functions per contract	28.1
Mean nodes per function	26.3
Mean LoC per function	5.1

Table 6. Vulnerability Type Distribution in the Synthetic Dataset

Vulnerability Type	Contracts
Clean (no vulnerability)	2,257
Cross-function (coupled)	983
tx_origin	186
timestamp	171
unchecked_send	170
overflow	146
underflow	111
unhandled_exception	109
reentrancy	75
Total	**4,208**

The vulnerability distribution across contract types is presented in Table 6. Cross-function (coupled) vulnerabilities constitute the largest injected category (983 contracts), followed by point-injected tx_origin (186), timestamp (171), and unchecked_send (170) vulnerabilities.

Graph Structure and Edge-Type Distribution. The heterogeneous ASG representation produces a rich multi-relational graph. Across the synthetic dataset's 4,208 contracts, the total graph contains 3,609,551 nodes and 4,680,370 edges spanning 6 active edge types. Table 7 details the edge-type distribution, partitioned by intra-function (within a single function subgraph) and inter-function (connecting nodes across different functions) scope. AST edges dominate (77.1%), capturing syntactic parent–child relationships; reference edges (17.0%) encode variable definition–use links; control-flow edges (`cfg_next`, `cfg_true`, `cfg_false`) account for 4.9%; and call edges (1.1%) represent function invocations. This composition ensures that the GNN receives both structural and semantic signals. Notably, 76.7% of edges are intra-function, supporting the HCA-SGNN's local-channel design,

Just Accepted

while the remaining 23.3% of inter-function edges provide the cross-function context captured by the global channel.

Table 7. Edge-Type Distribution in the Synthetic Dataset (4,208 contracts)

Edge Type	Intra-Function	Inter-Function	Total
ast	2,994,072	611,271	3,605,343
ref	377,875	416,206	794,081
cfg_next	196,162	9,191	205,353
call	—	51,004	51,004
cfg_true	17,742	820	18,562
cfg_false	5,999	28	6,027
Total	**3,591,850**	**1,088,520**	**4,680,370**

Template-Based Data Splitting. To prevent data leakage between training, validation, and test partitions, we adopt a *template-aware splitting* strategy. Standard random splitting risks placing contracts injected with the same vulnerability template into both training and evaluation sets, causing the model to memorize template-specific patterns rather than learning generalizable vulnerability semantics. Instead, we group contracts by their injection template and assign entire template groups to a single partition. Using a 70/15/15 split ratio over the 59 templates (36 point + 23 coupled), we ensure that no template appears in more than one partition. This design guarantees that the model is evaluated on vulnerability patterns it has never encountered during training, providing a rigorous assessment of its generalization capacity.

4.1.2 Real-World Dataset. For out-of-distribution evaluation, we compile a real-world benchmark of 242 contracts with 3,134 functions, sourced from the `SmartBugs Curated` [Durieux et al. 2020] and `SolidiFI` [Ghaleb and Pattabiraman 2020] repositories. Among these, 170 functions (5.42%) are labeled as vulnerable across 10 vulnerability categories. In `SmartBugs`, vulnerability locations are annotated directly in the source code using the `@vulnerable_at_lines` marker, which we parse to derive function-level labels. In `SolidiFI`, bug information is recorded in CSV logs containing line numbers, bug types, and span lengths; these annotations are mapped to function boundaries via AST analysis.

Table 8 summarizes the structural properties of the real-world dataset, and Table 9 presents its vulnerability type distribution.

Compared to the synthetic dataset (Table 5), the real-world contracts exhibit higher structural variance—the mean contract-level node count is lower (414.9 vs. 857.8) but the standard deviation is larger (549.7 vs. 518.0), reflecting the Real-World corpus's broader diversity in contract complexity ranging from small utility contracts (min. 12 nodes) to large multi-module contracts (max. 4,857 nodes). At the function level, the two datasets are closely aligned: the mean nodes per function (27.6 vs. 26.3) and LoC per function (5.7 vs. 5.1) are comparable, confirming that the fundamental unit of vulnerability analysis—individual functions—shares consistent structural properties across both synthetic and real-world settings.

4.1.3 Representativeness Analysis. A key concern for any synthetic dataset is whether it reflects the structural properties of real-world smart contracts or introduces sampling bias. To address this, we compare our synthetic corpus against the large-scale `Slither-Audited Smart Contracts` dataset [Rossini 2022], which contains 113,000 on-chain verified Ethereum smart contracts spanning 7.16 million functions across Solidity versions

Just Accepted

Table 8. Real-World Dataset Structural Overview

Metric	Value
Total contracts	242
Total functions	3,134
Vulnerable functions	170 (5.42%)
Clean functions	2,964 (94.58%)
Mean ASG nodes/contract	414.9
Mean ASG edges/contract	528.7
Mean functions/contract	13.0
Mean nodes/function	27.6
Mean LoC/function	5.7
Mean LoC/contract	96.4

Table 9. Vulnerability Type Distribution in the Real-World Dataset

Vulnerability Type	Contracts
Clean (no vulnerability)	100
unhandled_exception	52
reentrancy	31
access_control	17
overflow	15
bad_randomness	8
denial_of_service	6
timestamp	5
front_running	4
other	3
short_addresses	1
Total	**242**

0.4.x–0.8.x. Table 10 compares key structural metrics between our synthetic dataset and the Slither-Audited reference.

Several observations emerge from this comparison. First, at the *function level*, the two datasets exhibit closely aligned distributions: the median LoC(Lines of Code) per function is identical (4 lines) and the means are similar (5.1 vs. 7.8), with interquartile ranges of [3, 6] and [3, 7], respectively. This indicates that the fundamental unit of analysis—individual functions—is structurally representative of the broader Ethereum ecosystem. Second, at the *contract level*, our synthetic corpus exhibits smaller contracts (median 160 vs. 276 LoC) with fewer functions (median 26 vs. 45). This difference is attributable to our filtering criteria (Section 3.2.1), which excludes very large multi-file contracts with complex import hierarchies that are difficult to flatten reliably. However, the IQR overlap ([111, 236] vs. [140, 562]) confirms that our dataset spans a substantial portion of the complexity spectrum. Third, the higher means relative to medians in the Slither-Audited dataset (e.g., 406.4 vs. 276 for LoC/contract) indicate heavy right-skew from mega-contracts with thousands of lines, which are rare in practice and often consist of library code rather than application logic. Our filtering naturally removes these extreme cases, producing a more focused corpus of moderate-complexity application contracts that are more relevant for vulnerability analysis.

Table 10. Structural Comparison: Synthetic Dataset vs. Slither-Audited Reference

Metric	Synthetic		Slither-Audited	
	Median	Mean	Median	Mean
LoC per contract	160	187.5	276	406.4
Functions per contract	26	28.1	45	63.3
LoC per function	4	5.1	4	7.8
Bytes per function	140	215.7	183	399.7

Overall, the structural alignment at the function level—where vulnerability detection fundamentally operates—confirms that our synthetic dataset does not introduce significant sampling bias with respect to code complexity,

Just Accepted

and models trained on it can be expected to generalize to real-world smart contracts of comparable structure. Importantly, the minimum-function filter (>2 function definitions) applied during injection corpus construction does *not* bias the model toward multi-function contracts at inference time: HCA-SGNN operates on individual function subgraphs extracted from whatever contract is presented, meaning its learned representations are agnostic to the total number of functions in the enclosing contract. This is empirically confirmed by the real-world benchmark, where the minimum contract size is 1 function ($Q_1 = 3$, mean = 13.0), yet HCA-SGNN achieves a contract-level F1 of 0.7798—the highest among all evaluated models.

Short contract analysis. To further verify that the >2-function training filter does not create systematic bias, we quantify the prevalence of single-function contracts across all three datasets. In the 113K-contract Slither-Audited reference, only 702 contracts (0.62%) contain exactly one function, and 1,819 (1.61%) contain two or fewer—confirming that single-function contracts occupy a small tail of the Ethereum population ($Q_1 = 23$ functions). In our curated synthetic dataset, 24 of 4,208 contracts (0.57%) have exactly one function; these arise from clean contracts where our AST parser identifies only a single `FunctionDefinition` node (e.g., minimal proxy contracts or single-purpose utility contracts). The filter therefore excludes less than 1% of representative contracts.

Critically, the real-world benchmark—which is *not* filtered by function count—contains 28 single-function contracts (11.6% of 242), of which 27 are vulnerable (spanning 8 categories: overflow 9, unhandled_exception 11, reentrancy 1, access_control 1, dos 2, bad_randomness 1, timestamp 1, other 1). Additionally, 48 contracts (19.8%) have $\leq$2 functions and 68 (28.1%) have $\leq$3 functions. Despite the training set containing very few such contracts, HCA-SGNN processes them without modification because it classifies each function subgraph independently. The model's overall F1 of 0.7798 on this unrestricted benchmark—which includes these small contracts—demonstrates that the training filter does not impair generalization to minimal contract structures.

4.2 Vulnerability Detection Task

We evaluate our HCA-SGNN model on both the synthetic and real-world datasets described above. For all contracts, function-level labels and Abstract Syntax Graphs (ASGs) are extracted using the pipeline detailed in Section 3.2.1 to enable fine-grained analysis.

4.2.1 Large Language Models vs. HCA-SGNN on Function-Level Detection. To contextualize the effectiveness of our *function-level* HCA-SGNN, we benchmarked it against three state-of-the-art large language models (LLMs) fine-tuned for vulnerability detection: **Llama-3.1-8B-Instruct** [Meta AI 2024], **Qwen2.5-7B-Instruct** [Qwen et al. 2025], and **Qwen2.5-Coder-7B-Instruct** [Hui et al. 2024]. Rather than relying on zero-shot prompting, which prior work has shown to be unreliable for fine-grained vulnerability classification, we *fine-tuned* each model on the same synthetic training set used by HCA-SGNN. Specifically, we applied Low-Rank Adaptation (LoRA) [Hu et al. 2021] with rank 8 for supervised fine-tuning (SFT) using LLaMA-Factory [Y. Zheng et al. 2024]. Each model was trained for 3 epochs with a learning rate of 10^{-4}, cosine scheduler with 10% warmup, gradient accumulation of 8 steps, BF16 precision, and a context window of 2,048 tokens. Training data was prepared in ShareGPT format, where each function is presented individually with a zero-shot instruction, as shown in Figure 9.

At inference time, each function is evaluated independently. Contract-level predictions are derived by the same aggregation rule used for HCA-SGNN: a contract is labeled vulnerable if *any* of its functions is predicted vulnerable. We evaluate all models on both the held-out synthetic test set (631 contracts, 17,328 functions) and the real-world test set (242 contracts, 3,066 functions). Table 11 reports function-level (micro) metrics, and Table 12 reports contract-level (macro) metrics.

Function-Level Analysis. The function-level results (Table 11) reveal a stark performance gap between HCA-SGNN and the fine-tuned LLMs. On the synthetic test set, HCA-SGNN achieves an F1-score of **0.8258** with a

Just Accepted

Instruction prompt for function-level classification

Is the following Solidity smart contract vulnerable? Answer with 'vulnerable' or 'clean'.
[REDACTED SOLIDITY FUNCTION]

Fig. 9. Instruction prompt used for fine-tuning and evaluating LLMs on function-level vulnerability classification.

Table 11. Function-level (micro) detection performance of fine-tuned LLMs vs. HCA-SGNN. Each function is an independent sample.

Model	Synthetic Test			Real-World Test		
	Prec.	Rec.	F1	Prec.	Rec.	F1
HCA-SGNN (ours)	**0.7042**	**0.9979**	**0.8258**	**0.3609**	0.4882	**0.4150**
Llama-3.1-8B-Instruct [Meta AI 2024]	0.0340	0.5137	0.0638	0.1227	0.7765	0.2119
Qwen2.5-7B-Instruct [Qwen et al. 2025]	0.0248	0.4611	0.0472	0.1067	**0.8294**	0.1891
Qwen2.5-Coder-7B-Instruct [Hui et al. 2024]	0.0282	0.7579	0.0544	0.0596	0.6059	0.1085

Table 12. Contract-level (macro) detection performance of fine-tuned LLMs vs. HCA-SGNN. A contract is predicted vulnerable if any constituent function is predicted vulnerable.

Model	Synthetic Test			Real-World Test		
	Prec.	Rec.	F1	Prec.	Rec.	F1
HCA-SGNN (ours)	**0.7812**	**1.0000**	**0.8772**	**0.8000**	0.7606	**0.7798**
Llama-3.1-8B-Instruct [Meta AI 2024]	0.4872	1.0000	0.6552	0.5576	**0.8521**	0.6741
Qwen2.5-7B-Instruct [Qwen et al. 2025]	0.4880	1.0000	0.6559	0.5644	0.8944	0.6921
Qwen2.5-Coder-7B-Instruct [Hui et al. 2024]	0.4856	1.0000	0.6538	0.5196	0.7465	0.6127

precision of 0.7042 and near-perfect recall (0.9979), indicating that it correctly identifies vulnerable functions while maintaining a manageable false-positive rate. In contrast, all three LLMs exhibit extremely low precision (0.0248–0.0340) despite moderate-to-high recall (0.4611–0.7579), resulting in F1-scores below 0.07. This means the LLMs flag the vast majority of functions—both vulnerable and benign—as vulnerable, producing an overwhelming number of false positives that render their function-level predictions practically unusable.

On the real-world test set, the same pattern persists. HCA-SGNN achieves the highest F1 (0.4150) with substantially higher precision (0.3609) than any LLM. The LLMs achieve higher recall on real-world data (0.6059–0.8294) than on synthetic data, yet their precision remains critically low (0.0596–0.1227), yielding F1-scores of 0.1085–0.2119. Notably, **Qwen2.5-Coder-7B-Instruct**, despite being a code-specialized model, produces the lowest F1 on both test sets, suggesting that general code understanding capabilities do not directly transfer to fine-grained vulnerability localization.

Contract-Level Analysis. The contract-level aggregation (Table 12) substantially narrows the performance gap. On the synthetic test set, all four models achieve perfect recall (1.0000), reflecting that every contract containing at least one vulnerable function is correctly flagged. However, HCA-SGNN's superior function-level precision

Just Accepted

translates to a much higher contract-level precision (0.7812 vs. 0.4856–0.4880), yielding an F1 of **0.8772** compared to 0.6538–0.6559 for the LLMs. The LLMs' contract-level precision hovers near the dataset's vulnerable-class prior (~48%), confirming that they effectively predict *every* contract as vulnerable.

On the real-world test set, this gap persists: HCA-SGNN achieves an F1 of **0.7798** with precision 0.8000, while the best-performing LLM (**Qwen2.5-7B-Instruct**) reaches an F1 of 0.6921 with precision 0.5644 and recall 0.8944. The LLMs' high recall at the contract level is an artifact of their aggressive function-level over-prediction: since virtually every function is flagged as vulnerable, every contract is consequently predicted as vulnerable, yielding high recall but low precision.

Key Findings. Three conclusions emerge from this comparison. First, fine-tuning does not resolve the precision deficit of LLMs at the function level. Despite training on the same vulnerability-labeled data, the LLMs fail to learn discriminative function-level features, instead defaulting to a near-constant "vulnerable" prediction. Second, **HCA-SGNN**'s graph-based structural reasoning provides a decisive advantage for function-level vulnerability localization: by encoding the Abstract Syntax Graph topology and leveraging dual-channel message passing with cross-attention, HCA-SGNN captures fine-grained code patterns that text-based models cannot. Third, contract-level aggregation masks function-level deficiencies: the LLMs appear moderately competitive at the contract level (F1 ≈ 0.65–0.69), but this performance is driven entirely by their tendency to flag nearly all functions as vulnerable, a strategy that cannot guide developers to the specific vulnerable code location. In practice, a model that identifies every function as vulnerable provides no actionable insight, underscoring the importance of function-level precision for real-world deployment.

Discussion: GNN vs. LLM Trade-offs. Beyond the empirical results above, the choice of a GNN-based architecture for the detection task is motivated by several complementary considerations. *First, model size and deployment efficiency:* HCA-SGNN contains approximately 1 M trainable parameters—roughly 7,000× fewer than the 7–8 B-parameter LLMs benchmarked above. This compactness translates to training in minutes on a single GPU and negligible memory at inference, enabling integration into CI/CD pipelines, IDE plugins, and other resource-constrained developer toolchains where deploying a multi-billion-parameter model is impractical. *Second, structural inductive bias:* smart-contract vulnerability patterns are fundamentally defined by structural relationships—control-flow ordering (e.g., state update after external call in reentrancy), data-flow dependencies (e.g., unchecked return values), and cross-function state mutations. GNNs operate directly on these relational structures via message passing over typed AST, CFG, and data-flow edges, whereas LLMs must re-learn such dependencies implicitly from flattened token sequences and positional encodings—a representational mismatch that our precision results quantify. *Third, domain specificity:* Solidity constitutes a small fraction of LLM pre-training corpora, and its unique constructs (`msg.sender`, `payable` modifiers, re-entrancy guards, `delegatecall`, storage-vs-memory semantics) have limited representation. HCA-SGNN, by contrast, learns entirely from task-specific structural representations and does not depend on pre-trained token-level knowledge of any particular language, making it more robust for niche domains. *Finally*, we note that our overall framework already adopts a *hybrid* GNN+LLM design: HCA-SGNN handles detection (where structural reasoning and efficiency are paramount), while fine-tuned open-source LLMs (Llama-3.1-8B-Instruct, Qwen2.5-7B, Qwen2.5-Coder-7B) generate structured natural-language explanations (where fluency and semantic understanding matter most). This division of labor exploits the complementary strengths of each paradigm rather than relying on a single model for both tasks.

4.2.2 Contract-Level Vulnerability Detection Benchmark. Most state-of-the-art GNN baselines for smart-contract analysis operate at the *contract* level rather than at the *function* level. To enable a fair comparison with these baselines, we aggregate our function-level HCA-SGNN predictions to the contract level: a contract is labeled as *vulnerable* if *any* of its constituent functions is predicted vulnerable.[4] We benchmark ten models spanning diverse

[4]This aggregation reflects industry practice, where a single exploitable function compromises the entire contract.

Just Accepted

architectural paradigms: our function-level HCA-SGNN, classical message-passing networks (GAT [Veličković et al. 2018]), temporal and relational GNNs (DR-GCN, TMP [Yuan Zhuang et al. 2020]), graph transformers (TransformerGNN [P. Zhang et al. 2024]), edge-type-aware architectures (SCVHUNTER [Luo et al. 2024], ML-AGNN [H. Li et al. 2024]), subgraph-based detectors (BugSweeper, BugSweeperLight [Lee et al. 2025]), and bytecode-semantic hybrids (BSGVD [C. Xu et al. 2025]). All contract-level baselines are trained on the same template-split synthetic training set (2,946 graphs) for 30 epochs with identical hyperparameters (hidden dimension 128, 4 layers, 4 attention heads, dropout 0.1, learning rate 10^{-3}, batch size 32). HCA-SGNN is trained with its own optimized hyperparameters (hidden dimension 128, 4 layers, 8 attention heads, dropout 0.2, learning rate 5×10^{-4}, 20 epochs). All models are evaluated on both the held-out *synthetic test set* and the entirely held-out *real-world test set* (242 contracts from SmartBugs Curated and SolidiFI). Table 13 and Table 14 report the results.

Baseline Selection Rationale. Our baseline selection follows the controlled-experiment principle [Wohlin et al. 2012]: all models must be trainable on the *same* labelled dataset and evaluable on the *same* held-out test sets to allow fair comparison. Two recently proposed methods—SmartBugBert [Bu et al. 2025] and SmartVD [Alam et al. 2024]—are related but cannot be integrated into our benchmark under this protocol for the following reasons:

- *SmartBugBert* operates on **EVM bytecode**: it decompiles deployed contracts into opcode sequences, extracts TF-IDF features, constructs bytecode-level CFGs, and feeds them to a BERT + LightGBM classifier. Our pipeline, by contrast, starts from Solidity *source code* and builds AST-derived heterogeneous graphs. The two approaches consume fundamentally different input modalities, use different feature extractors, and target different deployment scenarios (bytecode-only vs. source-available contracts). Furthermore, SmartBugBert's vulnerability taxonomy (transaction-ordering, access control, self-destruct, timestamp dependency) only partially overlaps with ours, and no publicly available re-implementation or pre-trained model exists at the time of writing, making reproduction on our dataset infeasible. We note that our benchmark already includes BSGVD [C. Xu et al. 2025], a bytecode-semantic hybrid GNN baseline, which represents the bytecode-leveraging paradigm.
- *SmartVD* benchmarks LLMs (CodeLlama, Llama2, CodeT5, Falcon, GPT-3.5 Turbo, GPT-4o Mini) on their proprietary VulSmart dataset using the OWASP Top Ten taxonomy for *multi-class* classification. Our evaluation protocol uses *binary* classification (vulnerable vs. clean) with function-level labels on our synthetic and real-world test sets. The differing datasets, label taxonomies, and task formulations make direct numerical comparison misleading. To represent the LLM paradigm under controlled conditions, we fine-tuned three state-of-the-art LLMs (Llama-3.1-8B-Instruct, Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct) on the *same* training data as HCA-SGNN (Section 4.2.1), providing a methodologically rigorous LLM comparison.
- *ReEP* [Z. Wang et al. 2025] and *AWDNN* [Osei et al. 2025] are **reentrancy-specialized** detectors that target a single vulnerability type and cannot be trained on our eight-type synthetic corpus under the same controlled protocol. Including them alongside multi-type detectors in Tables 13–14 would conflate detection scope with detection quality. Instead, we conduct a dedicated reentrancy-specific evaluation in which all benchmarked models are re-evaluated on a reentrancy-only subset alongside AWDNN; these results appear in Appendix A.4. ReEP's released artifact additionally presents a reproducibility challenge: the ensemble voting layer depends on pre-computed outputs from six static analysis tools tied to the authors' original evaluation set, making a faithful re-evaluation on our dataset infeasible within the scope of this study.

Synthetic Test Set Analysis. On the synthetic test set, our **HCA-SGNN** achieves the highest F1-score (0.8772) and perfect recall (1.0), demonstrating that its function-level predictions, when aggregated to the contract level, detect every vulnerable contract in the test set. In absolute terms, HCA-SGNN correctly identifies all 300 vulnerable

Just Accepted

Table 13. Contract-level performance on the Synthetic Test Set. HCA-SGNN operates at the function level and aggregates predictions to the contract level via the "any vulnerable function" rule.

Model	Granularity	F1	Recall	Precision	Accuracy	AUC
HCA-SGNN (ours)	Function	**0.8772**	**1.0000**	0.7812	0.8669	**0.9942**
SCVHUNTER [Luo et al. 2024]	Contract	0.8295	0.7300	**0.9605**	**0.8576**	0.9609
DR-GCN [Yuan Zhuang et al. 2020]	Contract	0.7176	0.7833	0.6620	0.7073	0.7646
BSGVD [C. Xu et al. 2025]	Contract	0.7173	0.7867	0.6592	0.7057	0.7624
BugSweeper [Lee et al. 2025]	Function	0.7127	0.7567	0.6736	0.7104	0.7801
BugSweeperLight [Lee et al. 2025]	Function	0.7073	0.7933	0.6381	0.6883	0.7476
GAT [Veličković et al. 2018]	Contract	0.6990	0.6733	0.7266	0.7247	0.7716
TMP [Yuan Zhuang et al. 2020]	Contract	0.6911	0.7233	0.6616	0.6930	0.7598
TransformerGNN [P. Zhang et al. 2024]	Contract	0.6746	0.6633	0.6862	0.6962	0.7526
ML-AGNN [H. Li et al. 2024]	Contract	0.6656	0.6933	0.6400	0.6693	0.7353

Table 14. Contract-level performance on the Real-World Test Set (242 contracts).

Model	Granularity	F1	Recall	Precision	Accuracy	AUC
HCA-SGNN (ours)	Function	**0.7798**	**0.7606**	0.8000	**0.7479**	**0.7454**
SCVHUNTER [Luo et al. 2024]	Contract	0.6636	0.5141	**0.9359**	0.6942	0.6980
BugSweeperLight [Lee et al. 2025]	Function	0.5211	0.4789	0.5714	0.4835	0.4599
BSGVD [C. Xu et al. 2025]	Contract	0.4583	0.3873	0.5612	0.4628	0.4653
DR-GCN [Yuan Zhuang et al. 2020]	Contract	0.4034	0.3310	0.5165	0.4256	0.3942
ML-AGNN [H. Li et al. 2024]	Contract	0.2476	0.1831	0.3824	0.3471	0.3242
BugSweeper [Lee et al. 2025]	Function	0.2347	0.1761	0.3521	0.3264	0.3324
TransformerGNN [P. Zhang et al. 2024]	Contract	0.2300	0.1620	0.3966	0.3636	0.2634
GAT [Veličković et al. 2018]	Contract	0.2292	0.1549	0.4400	0.3884	0.3745
TMP [Yuan Zhuang et al. 2020]	Contract	0.2233	0.1620	0.3594	0.3388	0.3413

contracts (TP = 300, FN = 0) with 84 false positives out of 331 clean contracts, yielding a precision of 0.7812. At the function level, the model correctly flags 464 of 465 vulnerable functions (recall = 0.9979) with 195 false positives out of 16,863 clean functions, for a function-level precision of 0.7042. The model also attains the highest AUC (0.9942), reflecting excellent discrimination across classification thresholds. The moderate false-positive rate at the contract level is a consequence of the aggressive aggregation rule, where a single false-positive function prediction flags the entire contract. This trade-off is acceptable in security-critical applications where missed vulnerabilities are far more costly than false alarms.

Among the contract-level baselines, **SCVHUNTER** ranks second (F1 = 0.8295) with the highest precision (0.9605) and strong AUC (0.9609). Its edge-type-aware relational graph attention mechanism enables fine-grained discrimination between vulnerable and benign patterns, resulting in very few false positives.

The remaining contract-level baselines cluster within a narrow F1 band of 0.66–0.72. **DR-GCN** (0.7176) and **BSGVD** (0.7173) perform comparably, with DR-GCN benefiting from its degree-aware regularization and BSGVD leveraging fastText-based semantic embeddings. **BugSweeper** (0.7127) and **BugSweeperLight** (0.7073), both subgraph-aware architectures, achieve competitive recall (0.7567 and 0.7933 respectively). Classical architectures

Just Accepted

such as **GAT** (0.6990), **TMP** (0.6911), **TransformerGNN** (0.6746), and **ML-AGNN** (0.6656) trail behind, consistent with prior findings that homogeneous or weakly typed graph representations are insufficient for capturing the heterogeneous structural patterns characteristic of smart contract vulnerabilities.

Real-World Generalization. The real-world evaluation reveals a substantial performance gap across all contract-level baselines, reflecting the inherent difficulty of generalizing from synthetic to manually curated vulnerability benchmarks. Crucially, **HCA-SGNN** maintains its leading position with the highest F1 (0.7798), recall (0.7606), accuracy (0.7479), and AUC (0.7454). In absolute terms, HCA-SGNN correctly detects 108 of 142 vulnerable contracts (TP = 108, FN = 34) with only 27 false positives out of 100 clean contracts. At the function level, the model identifies 83 of 170 vulnerable functions (recall = 0.4882) with 147 false positives, yielding a function-level precision of 0.3609. Compared to the synthetic setting, HCA-SGNN's contract-level F1 decreases by only 9.7 percentage points (from 0.8772 to 0.7798), representing the smallest generalization gap among all evaluated models. This robustness stems from its function-level architecture: by learning vulnerability patterns at the granularity of individual functions rather than entire contracts, HCA-SGNN captures more transferable structural and semantic features that generalize beyond training-time injection templates.

SCVHUNTER ranks second in real-world F1 (0.6636) with the highest real-world precision (0.9359), but its recall drops sharply from 0.7300 to 0.5141, indicating that its high-precision predictions become overly conservative on out-of-distribution data. **BugSweeperLight** (F1 = 0.5211) is the only other model surpassing 0.50 F1, with its lightweight architecture (188K parameters) being less prone to overfitting on synthetic patterns than its larger sibling BugSweeper (11.6M parameters, F1 = 0.2347).

The majority of contract-level baselines experience severe degradation on real-world data: **TMP** drops from 0.6911 to 0.2233, **TransformerGNN** from 0.6746 to 0.2300, and **GAT** from 0.6990 to 0.2292. This sharp decline highlights the distributional shift between injected vulnerabilities and organically occurring bugs, which differ in complexity, context, and interaction with surrounding contract logic.

Per-Vulnerability-Type Detection Breakdown. To provide deeper insight into detection capabilities across vulnerability categories, Table 15 reports HCA-SGNN's contract-level detection broken down by vulnerability type on the real-world test set. On the synthetic test set, HCA-SGNN achieves *perfect contract-level recall* (1.0) across all 8 vulnerability categories, indicating that it has fully learned the structural patterns of every injected vulnerability type.

Several patterns emerge from this breakdown. Reentrancy (96.8% recall) and unhandled exception (86.5%) achieve the highest recall among frequent types, consistent with their strong representation in the training data. Notably, `bad_randomness` (87.5%) and `front_running` (75.0%)—types *entirely absent* from the training data—are detected with high recall, demonstrating that HCA-SGNN learns generalizable structural vulnerability patterns rather than memorizing template-specific artifacts. Conversely, `overflow` (20.0%) and `access_control` (52.9%) show lower recall: real-world overflow vulnerabilities involve complex arithmetic patterns across multiple expressions that differ substantially from simple injection templates, while access control vulnerabilities require reasoning about authorization logic that may not manifest as distinctive graph patterns. The 34 missed vulnerable contracts (FN) are distributed primarily across overflow (12), access_control (8), and unhandled_exception (7).

Per-Vulnerability-Type Function-Level Detection. To assess whether HCA-SGNN can discriminate between vulnerable and benign functions within each vulnerability category, Table 16 reports function-level precision, recall, and F1 for every vulnerability type on both the synthetic and real-world benchmarks.

On the synthetic benchmark, several vulnerability types achieve *perfect or near-perfect function-level precision*, indicating that HCA-SGNN can distinguish vulnerable functions from structurally similar benign ones within the same contract. Most notably, **reentrancy** achieves precision = 1.000 and recall = 1.000: all 24 reentrancy-bearing functions are correctly identified, and *zero* benign functions are misclassified among the 378 clean functions

Just Accepted

Table 15. Per-vulnerability-type contract-level detection on the real-world test set. Types marked with † are *absent from the synthetic training data.*

Vulnerability Type	#Contracts	Detected (TP)	Recall
Reentrancy	31	30	0.968
Unhandled Exception	52	45	0.865
Bad Randomness†	8	7	0.875
Front Running†	4	3	0.750
Denial of Service	6	4	0.667
Timestamp Dependence	5	3	0.600
Access Control†	17	9	0.529
Overflow	15	3	0.200
Other†	3	3	1.000
Short Addresses†	1	1	1.000
Total	**142**	**108**	**0.761**

Table 16. Per-vulnerability-type *function-level* detection on the synthetic and real-world test sets. Types marked with † are absent from the synthetic training data. **Prec.** = precision, **Rec.** = recall. Clean rows report false-positive counts among non-vulnerable functions.

Vulnerability Type	Synthetic Test			Real-World Test		
	Prec.	Rec.	F1	Prec.	Rec.	F1
Reentrancy	1.000	1.000	1.000	0.539	0.903	0.675
Cross-Function	0.832	0.997	0.907	—	—	—
Overflow	0.968	1.000	0.984	0.750	0.143	0.240
Underflow	0.833	1.000	0.909	—	—	—
Timestamp	0.773	1.000	0.872	1.000	0.500	0.667
Tx_Origin	0.861	1.000	0.925	—	—	—
Unchecked Send	0.880	1.000	0.936	—	—	—
Unhandled Exception	1.000	1.000	1.000	0.417	0.522	0.464
Access Control†	—	—	—	0.211	0.211	0.211
Bad Randomness†	—	—	—	0.056	0.091	0.069
Front Running†	—	—	—	0.667	0.400	0.500
Denial of Service	—	—	—	0.600	0.500	0.546
Clean (FP only)	118 FP / 7,896 funcs (1.5%)			37 FP / 2,172 funcs (1.7%)		

co-located in those contracts. This means functions containing external calls *without* the characteristic "call → state update" reentrancy pattern are correctly classified as non-vulnerable. Similarly, **unhandled exception** (precision = 1.000) and **overflow** (precision = 0.968) demonstrate strong discriminative capability. On clean contracts, only 118 out of 7,896 non-vulnerable functions (1.5%) are false positives, confirming that the model does not indiscriminately flag functions that happen to contain common code patterns such as external calls, arithmetic operations, or `require` statements.

Just Accepted

On the real-world benchmark, precision is expectedly lower due to the distribution shift from synthetic to organic code. Reentrancy function-level recall remains high (0.903) with 28 of 31 vulnerable functions detected, though precision drops to 0.539 due to 24 false positives among real-world functions whose complex call patterns superficially resemble reentrancy structures. The ablation study (Table 17) further confirms that this discriminative capability stems primarily from the dual-channel architecture: removing the local/global channel separation degrades overall function-level precision from 0.7042 to 0.4207, nearly doubling the false-positive rate.

Key Findings. Three conclusions emerge from this benchmark. First, **function-level modeling provides a decisive advantage**: HCA-SGNN's function-level approach, when aggregated to the contract level, outperforms all contract-level baselines on both synthetic and real-world data, achieving the best F1 on both benchmarks (0.8772 and 0.7798). The function-level granularity not only enables precise vulnerability localization but also yields more robust contract-level predictions by grounding classification in fine-grained structural patterns. Second, **edge-type-aware architectures** that explicitly model the heterogeneous structure of smart contract ASGs (SCVHUNTER) consistently outperform homogeneous GNNs among the contract-level baselines.

Third, the **synthetic-to-real generalization gap** reflects a fundamental distinction between intra-type and cross-type generalization. On the synthetic benchmark, our template-aware data splitting (Section 3.2.1) ensures that the test set contains vulnerability *variants*—i.e., different templates of the same vulnerability types seen during training (e.g., `call.value`-based vs. `delegatecall`-based reentrancy). All models can thus be evaluated on their ability to generalize across implementation patterns within known vulnerability categories, and the strong synthetic performance confirms this intra-type generalization is achievable. However, the real-world dataset (Table 9) introduces entirely *unseen vulnerability types* absent from the synthetic training set—notably `bad_randomness` (8 contracts), `front_running` (4), `short_addresses` (1), and `other` (3)—which together account for 16 of the 142 vulnerable contracts. Since no training examples exist for these categories, models cannot learn their characteristic structural patterns, leading to inevitable misclassification. This explains why even HCA-SGNN, despite strong intra-type generalization, experiences a 9.7 percentage point F1 drop on real-world data, while contract-level baselines that lack fine-grained structural reasoning suffer far more severe degradation (40–70% F1 loss). In essence, the models successfully generalize from one reentrancy variant to another, but cannot transfer learned reentrancy patterns to detect an entirely different vulnerability class such as bad randomness. This highlights the importance of expanding the injection framework to cover additional vulnerability categories in future work, as well as the potential of few-shot and zero-shot transfer techniques for handling novel vulnerability types.

4.2.3 Ablation Study. To quantify the contribution of each architectural component in HCA-SGNN, we conduct a systematic ablation study. Starting from the full model configuration, we independently disable or replace each key component—dual-channel message passing, cross-attention, and pooling strategy—while keeping all other hyperparameters fixed (hidden dimension 128, 4 layers, 8 heads, dropout 0.2, learning rate 5×10^{-4}, 20 epochs). Table 17 reports both function-level and contract-level F1 scores on the synthetic and real-world test sets.

Dual-Channel Message Passing. Removing the dual-channel architecture—collapsing both global and local channels into a single-channel encoder operating on the full contract graph—produces the largest performance degradation of any ablation: function-level F1 drops by 23.6 percentage points (from 0.8258 to 0.5897). This severe decline confirms that the dual-channel design is the most critical architectural component. Without explicit separation of intra-function and inter-function message passing, the model cannot distinguish between local vulnerability patterns (e.g., unchecked arithmetic within a single function) and cross-function interactions (e.g., state shared between a setter and executor). The single-channel variant also exhibits sharply reduced precision (0.4207 vs. 0.7042), indicating that the loss of function-level locality leads to substantially more false positives.

Just Accepted

Table 17. Ablation study of HCA-SGNN components. Δ denotes the change in synthetic function-level F1 relative to the full model. **Bold** indicates the best value per column.

Variant	Synthetic Test		Real-World Test		Δ F1
	Func. F1	Cont. F1	Func. F1	Cont. F1	
HCA-SGNN (Full)	0.8258	0.8772	**0.4150**	0.7798	—
Architectural components					
↪ No Dual-Channel	0.5897	0.8005	0.4000	0.7476	−0.236
↪ No Cross-Attention	0.7977	0.8785	0.4273	**0.7944**	−0.028
Pooling strategy					
↪ Hierarchical Pool	**0.8722**	**0.9105**	0.3914	0.7287	+0.046
↪ Max Pool	0.8087	0.8889	0.3155	0.6667	−0.017
↪ Attention Pool	0.8079	0.8974	0.4240	0.7619	−0.018

Cross-Attention Between Functions. Disabling the inter-function cross-attention module reduces synthetic function-level F1 by 2.8 percentage points (from 0.8258 to 0.7977). The impact is modest on synthetic data because our template-aware splitting already separates vulnerability variants, and many point-injection vulnerabilities are self-contained within individual functions. Interestingly, the no-cross-attention variant achieves the highest real-world contract-level F1 (0.7944), slightly exceeding the full model (0.7798). This suggests that cross-attention, while beneficial for detecting coupled vulnerabilities in the synthetic setting, may introduce noise when applied to real-world contracts whose inter-function interaction patterns differ from the synthetic training distribution. Nevertheless, the cross-attention module remains valuable for the coupled vulnerability class, which constitutes the largest category in our synthetic dataset (983 contracts).

Pooling Strategy. Among the three alternative pooling strategies, **hierarchical multi-scale pooling** achieves the highest synthetic function-level F1 (0.8722, +4.6 pp over the full model's mean pooling), combining mean, max, and importance-weighted aggregation with learned scale coefficients. However, this advantage does not transfer to real-world data (function-level F1 = 0.3914 vs. 0.4150), suggesting that the additional pooling parameters overfit to synthetic patterns. **Max pooling** (F1 = 0.8087) and **attention pooling** (F1 = 0.8079) perform comparably on synthetic data, both slightly below the full model. On real-world data, attention pooling achieves the best function-level F1 among the pooling variants (0.4240), consistent with its ability to dynamically focus on the most informative nodes. Mean pooling (the full model's default) offers the best balance between synthetic and real-world performance, making it the preferred default configuration.

Summary. The ablation study reveals a clear hierarchy of component importance: (1) dual-channel message passing is indispensable (−23.6 pp), confirming that explicit separation of intra- and inter-function context is the defining feature of HCA-SGNN; (2) cross-attention contributes meaningfully (−2.8 pp), particularly for coupled vulnerability detection; and (3) the choice of pooling strategy modulates the precision–generalization trade-off, with mean pooling providing the most robust default.

4.2.4 Overfitting Analysis and Cross-Dataset Evaluation. A central concern when training on synthetically injected data is whether models overfit to injection-specific artifacts rather than learning genuine vulnerability patterns. We address this through four complementary analyses.

Just Accepted

Train–Test Convergence Indicators. All models are trained with early stopping (patience 10–20 epochs) on validation loss. HCA-SGNN converges at epoch 20 with a training loss of 0.0148 and validation loss of 0.0297, yielding a loss ratio of 2.0×. While this indicates mild overfitting, the model maintains strong held-out performance (synthetic F1 = 0.8258 at the function level), suggesting that the learned representations remain generalizable. By contrast, larger-capacity models such as BugSweeper (11.6M parameters) exhibit a more pronounced synthetic-to-real-world F1 collapse (0.7127 → 0.2347, a 67% relative drop), consistent with overfitting to training-set specifics. BugSweeperLight, with only 188K parameters, retains substantially better real-world F1 (0.5211), confirming that model capacity is a direct overfitting risk factor.

Synthetic-to-Real-World Generalization Gap. Table 18 quantifies the generalization gap across all 10 benchmarked models by comparing contract-level F1 on the synthetic and real-world test sets. HCA-SGNN exhibits the smallest absolute F1 drop (9.7 pp), while the median baseline suffers a 36.6 pp drop.

Table 18. Synthetic-to-real-world generalization gap (contract-level F1). Models are sorted by real-world F1. Δ F1 = Synthetic F1 − Real-World F1. *Relative drop* = Δ F1 / Synthetic F1 × 100%.

Model	**Synth. F1**	**Real F1**	**Δ F1**	**Rel. Drop**
HCA-SGNN (ours)	0.8772	0.7798	0.097	11.1%
SCVHUNTER	0.8295	0.6636	0.166	20.0%
Hierarchical RGCN	0.7745	0.5893	0.185	23.9%
BugSweeperLight	0.7073	0.5211	0.186	26.3%
BSGVD	0.7173	0.4583	0.259	36.1%
DR-GCN	0.7176	0.4034	0.314	43.8%
ML-AGNN	0.6656	0.2476	0.418	62.8%
BugSweeper	0.7127	0.2347	0.478	67.1%
TransformerGNN	0.6746	0.2300	0.445	65.9%
GAT	0.6990	0.2292	0.470	67.2%
TMP	0.6911	0.2233	0.468	67.7%

Cross-Dataset Evaluation on Unseen Vulnerability Types. The strongest evidence against overfitting comes from HCA-SGNN's ability to detect vulnerability types *entirely absent from training data*. The real-world test set contains five such categories—`bad_randomness` (8 contracts), `front_running` (4), `access_control` (17), `short_addresses` (1), and `other` (3)—none of which appear in the synthetic training or validation sets. Despite having zero training examples for these categories, HCA-SGNN detects `bad_randomness` at 87.5% recall (7/8), `front_running` at 75.0% (3/4), and `access_control` at 52.9% (9/17). If the model had merely memorized injection template artifacts, it would have *zero* recall on these unseen types, since no such artifacts exist in the training data. The non-trivial detection rates therefore confirm that HCA-SGNN learns generalizable structural patterns—such as unguarded external calls, state inconsistencies, and control-flow anomalies—that transfer across vulnerability categories.

Template-Aware Splitting as an Anti-Overfitting Mechanism. Our template-aware data splitting (Section 3.2.1) assigns entire template groups to a single partition, ensuring that the test set contains vulnerability *variants* (different templates of the same type) that were never seen during training. Under standard random splitting, a model could achieve high test accuracy by recognizing template-level surface features (e.g., specific variable names or code structure common to a template). Our splitting strategy eliminates this shortcut, forcing the model to learn the underlying vulnerability semantics rather than template-specific patterns. The strong synthetic

Just Accepted

test performance (F1 = 0.8772) under this regime confirms that HCA-SGNN generalizes across implementation variants within the same vulnerability type.

4.2.5 Graph-Size Sensitivity Analysis. A well-known limitation of GNNs is that performance can degrade on larger graphs due to over-smoothing, where repeated message passing causes node representations to converge, and to signal dilution, where vulnerability-relevant nodes constitute a diminishing fraction of the total graph [Chen et al. 2020; G. Li et al. 2019]. Because smart contracts range from small utility libraries to large multi-module systems (12 to 4,857 ASG nodes in our datasets), it is important to verify that HCA-SGNN's performance remains robust across the full graph-size spectrum.

Methodology. We stratify the synthetic and real-world test sets into four quartile-based bins according to the number of ASG nodes per contract (Q1 = smallest 25%, Q4 = largest 25%) and compute both contract-level and function-level detection metrics within each bin. Table 19 and Table 20 report the results.

Table 19. Graph-size sensitivity on the **synthetic** test set (631 contracts). Bins are quartile-based by ASG node count. $\bar{n}$ = mean nodes per contract.

Bin	#C	#Vuln	$\bar{n}$	Contract-Level			Function-Level		
				Prec	Rec	F1	Prec	Rec	F1
Q1 (≤545)	158	33	330.6	0.559	1.000	0.717	0.566	1.000	0.723
Q2 (546–753)	158	61	643.1	0.782	1.000	0.878	0.729	1.000	0.843
Q3 (754–1,099)	159	93	899.5	0.809	1.000	0.894	0.789	1.000	0.882
Q4 (1,100–5,381)	156	113	1,603.4	0.856	1.000	0.923	0.678	0.995	0.807
Overall	631	300	857.8	0.781	1.000	0.877	0.704	0.998	0.826

Table 20. Graph-size sensitivity on the **real-world** test set (242 contracts). Bins are quartile-based by ASG node count. $\bar{n}$ = mean nodes per contract.

Bin	#C	#Vuln	$\bar{n}$	Contract-Level			Function-Level		
				Prec	Rec	F1	Prec	Rec	F1
Q1 (≤72)	62	55	44.9	0.967	0.527	0.682	0.794	0.436	0.563
Q2 (73–180)	61	57	119.9	0.982	0.930	0.955	0.456	0.590	0.514
Q3 (181–636)	59	21	397.5	0.750	0.857	0.800	0.372	0.500	0.427
Q4 (637–4,857)	60	9	1,114.3	0.296	0.889	0.444	0.054	0.267	0.090
Overall	242	142	414.9	0.800	0.761	0.780	0.361	0.488	0.415

Synthetic Test Set. On the synthetic benchmark, HCA-SGNN maintains perfect or near-perfect recall (1.000) across all four quartiles, demonstrating that the model successfully identifies vulnerable contracts regardless of graph size. The contract-level F1 score actually *increases* from Q1 (0.717) to Q4 (0.923), driven by improved precision: larger contracts contain more functions and thus more structural context, which helps the model distinguish genuine vulnerability patterns from benign code. Function-level F1 follows a similar ascending trend from Q1 (0.723) to Q3 (0.882) before a modest decline in Q4 (0.807), where the slight drop in recall (0.995) and

Just Accepted

precision (0.678) in the largest quartile suggests that the model occasionally flags benign functions or misses vulnerable ones in contracts exceeding 1,100 nodes. Importantly, this Q4 decline is mild—F1 remains above 0.80—and does not indicate systematic over-smoothing or failure.

Real-World Test Set. On the real-world benchmark, a different pattern emerges. The model achieves its strongest F1 in Q2 (0.955), where graph sizes of 73–180 nodes match the scale at which most real-world vulnerability-carrying contracts reside. In Q1 (≤72 nodes), high precision (0.967) but low recall (0.527) suggests that very small contracts—often minimal proxies or interface stubs—lack sufficient structural context for the model to recognise vulnerability patterns. In Q4 (637–4,857 nodes), precision drops sharply (0.296) while recall remains high (0.889), indicating that the model tends to over-predict vulnerabilities in large contracts. This is consistent with the observation that large real-world contracts contain more complex function interactions, producing graph structures that superficially resemble vulnerability patterns (e.g., cross-function state mutations), leading to false positives. The function-level F1 in Q4 (0.090) confirms that localising specific vulnerable functions within very large ASGs remains challenging, pointing to a productive direction for future architectural improvements.

Mitigation Strategies. Several design choices in HCA-SGNN inherently mitigate GNN scalability issues on large graphs:

(1) **Function-level subgraph decomposition.** By restricting the local message-passing channel to intra-function edges, HCA-SGNN bounds the effective receptive field for each function to its own ASG subgraph (mean 26.3 nodes). This prevents the over-smoothing that would occur if all $L = 4$ layers operated on the full contract graph, where the receptive field would grow to encompass most of the contract.
(2) **Dual-channel gated fusion.** The learned gate γ allows nodes in large contracts to down-weight noisy global signals, adaptively preserving local vulnerability patterns that might otherwise be diluted by irrelevant structural context from distant functions.
(3) **Hierarchical multi-scale pooling.** The importance-weighted component of the pooling mechanism assigns higher weights to discriminative nodes, ensuring that vulnerability-relevant nodes in large subgraphs are not drowned out by the majority of benign nodes.
(4) **Cross-attention between function embeddings.** Rather than relying solely on node-level message passing across the full graph (which scales poorly), HCA-SGNN lifts inter-function communication to the compressed function-embedding level, where the attention operates over G functions (typically 10–100) rather than $|\mathcal{V}|$ nodes (potentially thousands).

These mechanisms collectively ensure that HCA-SGNN's performance degrades gracefully rather than catastrophically as graph size increases. The empirical results confirm this: on the synthetic benchmark, function-level F1 remains above 0.80 even for the largest quartile (Q4: 1,100–5,381 nodes), and on the real-world benchmark, contract-level recall stays at 0.889 in Q4 despite the domain shift. Future work could further improve large-graph robustness through graph coarsening, adaptive message-passing depth, or attention-based neighbourhood sampling.

4.3 Vulnerability Explanation Task

4.3.1 Evaluation of LLM-as-Judge Reliability. Using LLMs as automated evaluators introduces the risk of systematic bias, particularly when the evaluator model is also one of the explanation generators. To assess the reliability of LLM-based scoring, we conducted two complementary analyses: a human correlation study and a multi-judge consistency analysis.

Human–LLM Correlation. We compared GPT-4o's automated scores against human expert annotations on 50 randomly selected explanations. Three dimensions were assessed: **Clarity**, **Identification**, and **Impact**.

Just Accepted

Agreement was measured using **Pearson correlation** (r) for linear association and **Kendall's Tau** (τ) for ordinal rank agreement, as reported in Table 21.

Table 21. Correlation between human expert and GPT-4o automated evaluations ($n = 50$).

	Clarity	**Identification**	**Impact**
Pearson r	0.100	0.860	0.321
Kendall's τ	0.062	0.708	0.310

GPT-4o demonstrates strong alignment with human judgement on the **Identification** dimension ($r = 0.860$, $\tau = 0.708$), confirming its reliability for evaluating the technical accuracy of vulnerability localisation and classification—the dimension most directly relevant to security auditing. The moderate correlation on **Impact** ($r = 0.321$, $\tau = 0.310$) reflects partial agreement on consequence assessment, with divergence likely arising from differing assumptions about attack feasibility and economic context. The near-zero correlation on **Clarity** ($r = 0.100$, $\tau = 0.062$) indicates that linguistic quality evaluation is inherently subjective and resists automated assessment, consistent with broader findings in NLG evaluation research [Y. Liu et al. 2023].

Multi-Judge Consensus as Bias Mitigation. To complement the human correlation study above, we additionally examine inter-judge consistency by having all three evaluator models score the same set of explanations independently. We defer the detailed per-judge score breakdowns to Table 24 in Section 4.3.2, but summarise the key structural finding here: despite systematic calibration offsets between judges (up to ∼0.67 points), the *relative ranking* of explanation models is preserved across all three evaluators—no model's position changes by more than one rank (Kendall's $\tau > 0.90$ between any judge pair). This consistency provides strong evidence that our multi-judge averaging protocol effectively neutralises individual evaluator biases, yielding a composite score that reflects explanation quality rather than evaluator idiosyncrasies.

Implications for Evaluation Design. These findings suggest a principled evaluation strategy: **Identification**—the dimension with the highest human–LLM agreement and the greatest practical relevance for vulnerability analysis—can be reliably assessed by automated LLM judges. **Impact** benefits from multi-judge averaging to mitigate moderate inter-rater variability. **Clarity**, being highly subjective, is best evaluated through human review when resources permit, or through multi-judge consensus as an approximation. The multi-judge protocol employed in this work addresses the well-documented self-enhancement bias [L. Zheng et al. 2023] whereby a model tends to rate its own outputs more favourably: by averaging across three independent judges from different providers, no single model's bias dominates the final scores.

Limitations and Future Work for Clarity and Impact Assessment. We acknowledge that the low human–LLM correlation on Clarity ($r = 0.100$) and moderate correlation on Impact ($r = 0.321$) remain open limitations of our evaluation protocol. Despite employing three independent judges from different providers, we were unable to identify an LLM-based evaluator that achieves strong human agreement on these two dimensions. As a partial mitigation, we adopt a *confidence-graded interpretation* throughout the benchmark analysis (Section 4.3.2): primary conclusions are grounded on Identification ($r = 0.860$), the most reliable dimension, while Clarity and Impact scores are reported with explicit caveats. Additionally, the multi-judge consensus ranking (Kendall's $\tau > 0.90$ between judge pairs) provides a robustness check: the *relative ordering* of models is stable across all three judges, suggesting that model rankings—if not absolute values—capture genuine quality differences even on the weaker dimensions.

Developing more reliable automated evaluation for Clarity and Impact remains an important direction for future work. Promising approaches include: (1) structured human-evaluation rubrics with domain-expert panels

Just Accepted

and anchor examples, which could establish a more reliable ground truth; (2) pairwise preference judgements ("Is explanation A clearer than explanation B?"), which mitigate the calibration issues of absolute Likert scoring and typically achieve higher inter-rater reliability; (3) dimension-specific judge fine-tuning on human-annotated explanation quality data; and (4) retrieval-augmented generation (RAG) with curated exploit databases to improve the *explanations themselves* on Impact, the consistently lowest-scored dimension across all model classes.

4.3.2 Explanation Performance Benchmark. We conduct a comprehensive benchmark of vulnerability explanation quality across eleven models spanning three paradigm categories: (1) *commercial API models*—GPT-4o [OpenAI 2024], Claude-3.5-Haiku [Anthropic 2024], and Gemini-2.5-Flash [Google DeepMind et al. 2024]; (2) *open-source base models*—Llama-3.1-8B [Meta AI 2024], Qwen2.5-7B [Qwen et al. 2025], DeepSeek-Coder-V2-16B [DeepSeek-AI et al. 2024], DeepSeek-R1-8B [Guo, Yang, et al. 2025], and GPT-OSS-20B; and (3) *fine-tuned open-source models*—Llama-3.1-8B-Instruct-FT, Qwen2.5-7B-FT, and Qwen2.5-Coder-7B-FT. The fine-tuned variants were obtained via LoRA-based supervised fine-tuning (rank 16, $\alpha = 32$, learning rate 10^{-4}, 5 epochs, cutoff length 4,096 tokens) on explanation data distilled from commercial API outputs, using LLaMA-Factory [Y. Zheng et al. 2024]. This design enables systematic comparison of explanation quality across model scale, architecture, and the effect of domain-specific knowledge distillation.

Evaluation Corpus and Protocol. Each model generated structured explanations for 197 vulnerable Solidity functions sampled from our synthetic dataset, spanning seven vulnerability categories: `overflow` (29), `reentrancy` (15), `timestamp` (26), `tx_origin` (46), `unchecked_send` (34), `underflow` (21), and `unhandled_exception` (26). All models received the same concise prompt template instructing them to produce explanations conforming to the five-field schema: *Bug Name*, *Affected Area*, *Description*, *Impact Assessment*, and *Mitigation*. Open-source base models were served via Ollama through an OpenAI-compatible API endpoint, while fine-tuned models used merged LoRA adapters served through vLLM.

Structural and Semantic Validation Results. Before applying LLM-judge scoring, we ran the rule-based validation checks described in Section 3.5.1 on all generated explanations. Table 22 reports the results.
Structural completeness is high across most models: 8 of 11 models achieve ≥98.4% structural completeness, confirming that the prompt-based schema enforcement is effective for well-tuned models. The two notable exceptions are **Gemini-2.5-Flash** (53.8%), which frequently omits the Impact Assessment and Mitigation fields in favour of a free-form narrative, and **DeepSeek-R1-8B** (72.0%), whose reasoning-oriented architecture struggles with the structured output format (as also reflected in its low yield of 25/197 non-empty outputs).

Semantic validation reveals that the most reliable check is *Description substantiveness*: 9 of 11 models achieve 100%, confirming that all models generate analytically meaningful text when they produce output. *Bug Name alignment*—the check most directly related to factual correctness—ranges from 84.0% (DeepSeek-R1-8B) to 100% (DeepSeek-Coder-V2-16B), with the fine-tuned models clustering at 87–88%. The 10–13% misalignment rate in fine-tuned models arises from a mix of (a) synonym usage (e.g., "Integer Overflow" vs. the canonical label "overflow"), (b) compound naming where the model identifies multiple co-occurring issues (e.g., "Unchecked Call Return Value & Reentrancy" for an `unhandled_exception` label), and (c) a small number of genuine category confusions. Importantly, even on misaligned samples, the LLM-judge Identification scores remain above average (cross-judge mean ≈ 3.7 on a 1–5 scale), suggesting that the explanations often capture the vulnerability semantics correctly despite the lexical mismatch in the Bug Name field. The *All Pass* rate—requiring all five semantic checks to hold simultaneously—exceeds 85% for all commercial and fine-tuned models except Gemini-2.5-Flash (49.7%), providing evidence that the generated explanations satisfy necessary conditions for structural well-formedness, semantic grounding, and practical actionability across the large majority of samples.

Validation Example. To illustrate the validation checks in practice, Figure 10 presents two representative cases: a *passing* GPT-4o explanation and a *failing* Gemini-2.5-Flash explanation.

Just Accepted

Table 22. Structural and semantic validation of generated explanations across all eleven models. **Structural completeness** measures the percentage of non-empty explanations containing all five required fields. **Semantic checks** verify: Bug Name alignment with the ground-truth vulnerability label, Affected Area referencing identifiers from the source code, Description substantiveness (≥10 words), Impact Assessment citing concrete consequences, and Mitigation providing actionable recommendations. **All Pass** indicates the fraction satisfying all five semantic checks simultaneously.

Model	Structural		Semantic Checks (%)					All
	n	Complete	Bug	Code	Desc.	Impact	Mitig.	Pass
Commercial API Models								
GPT-4o	197	100.0%	91.4	97.5	100.0	100.0	97.0	85.8
Claude-3.5-Haiku	197	99.5%	89.3	98.0	99.5	99.5	99.5	87.8
Gemini-2.5-Flash	197	53.8%	91.4	97.0	85.3	65.5	53.8	49.7
Fine-Tuned Open-Source Models (7–8B)								
Llama-3.1-8B-FT	195	100.0%	87.2	99.0	100.0	100.0	99.5	86.2
Qwen2.5-7B-FT	195	99.5%	88.2	99.0	100.0	99.5	99.0	86.7
Qwen2.5-Coder-7B-FT	195	100.0%	87.2	99.5	100.0	100.0	100.0	87.2
Open-Source Base Models								
GPT-OSS-20B	188	98.4%	97.9	98.4	99.5	98.9	98.4	95.7
DeepSeek-Coder-V2-16B	197	100.0%	100.0	99.0	100.0	100.0	100.0	99.0
DeepSeek-R1-8B[†]	25	72.0%	84.0	88.0	80.0	76.0	72.0	60.0
Qwen2.5-7B	197	100.0%	97.5	88.8	100.0	100.0	100.0	86.8
Llama-3.1-8B	197	99.5%	94.4	95.4	100.0	100.0	100.0	89.8

[†]Only 25 of 197 explanations were non-empty.

Multi-Judge Evaluation Protocol. A known limitation of using a single LLM as judge is systematic scoring bias [L. Zheng et al. 2023]. To mitigate this, we employ a *multi-judge* protocol in which *three* independent evaluator models—GPT-4o (OpenAI), Claude-3.5-Haiku (Anthropic), and Gemini-2.5-Flash (Google)—independently score each explanation on the three evaluation dimensions: **Clarity** (linguistic quality and logical structure), **Identification** (accuracy of vulnerability localization and root-cause analysis), and **Impact** (thoroughness of consequence assessment and remediation). Each evaluator assigns integer scores on a 1–5 Likert scale using dimension-specific Chain-of-Thought prompts. The final score for each model–dimension pair is computed as the arithmetic mean across the three judges, reducing the influence of any single evaluator's systematic bias. This multi-judge approach constitutes a substantial methodological improvement over single-evaluator protocols, as it provides a more robust and balanced assessment.

Results. Table 23 reports the mean scores per dimension and the overall average across all three evaluators for each explanation model. We additionally report per-evaluator breakdowns in Table 24 to quantify inter-judge variability. When interpreting these scores, the reader should bear in mind the per-dimension reliability findings from Section 4.3.1: **Identification** scores carry the highest confidence (Pearson $r = 0.860$ with human judgements), **Impact** scores carry moderate confidence ($r = 0.321$), and **Clarity** scores should be treated as approximate indicators rather than precise measures ($r = 0.100$). Accordingly, we ground our primary conclusions on the Identification dimension and treat Clarity and Impact as supplementary signals.

Commercial API Models. Among the commercial models, **GPT-4o** achieves the highest overall average (3.822) and leads in **Identification** (4.351)—the dimension with the strongest human–LLM agreement (Table 21)—confirming

Just Accepted

Passing example (GPT-4o, function `removePauser`, ground-truth: `tx_origin`)

Bug Name: Transaction Origin (tx.origin) Vulnerability
Affected Area: `require(tx.origin == controller);` in the `removePauser` function.
Description: The function uses `tx.origin` to verify the transaction initiator. `tx.origin` returns the original external account that started the transaction, which can be different from `msg.sender` if the call is made through a contract. This can be exploited in a phishing attack…
Impact Assessment: An attacker can potentially remove any pauser by tricking the controller into interacting with a malicious contract, leading to unauthorized access…
Mitigation: Replace `tx.origin` with `msg.sender` to ensure that only direct calls from the controller account are authorized…

Validation outcome — all checks pass:
✓ Structural: all 5 fields present. ✓ Bug Name: "Transaction Origin" matches alias for `tx_origin`.
✓ Code Ref: `tx.origin`, `controller` found in source. ✓ Description: 50+ words.
✓ Impact: cites "unauthorized access." ✓ Mitigation: recommends "Replace `tx.origin`."

Failing example (Gemini-2.5-Flash, function `requestWithdrawal`, ground-truth: `timestamp`)

Bug Name: Timestamp Dependence / Miner Manipulation
Affected Area: `if (block.timestamp == count)` condition within the `requestWithdrawal` function.
Description: The contract uses `block.timestamp == count` as a condition to trigger a 1 wei…
[output truncated by model—no Impact Assessment or Mitigation fields]

Validation outcome — structural failure:
× Structural: only 3/5 fields present (missing Impact Assessment, Mitigation).
✓ Bug Name: "Timestamp" matches `timestamp`. ✓ Code Ref: `block.timestamp`, `count` found.
× Impact: field absent. × Mitigation: field absent.

Fig. 10. Two representative validation examples from the benchmark data. The GPT-4o explanation passes all structural and semantic checks. The Gemini-2.5-Flash explanation is flagged as structurally incomplete—the model reorganised its output into a free-form narrative, omitting the Impact Assessment and Mitigation fields. Such failures typify the 46.2% structural failure rate observed for Gemini-2.5-Flash in Table 22.

Just Accepted

its strong capacity for precise vulnerability localization and root-cause analysis with high-confidence scoring. **Claude-3.5-Haiku** ranks closely behind (3.774) and achieves the highest **Clarity** score (4.022) across all commercial models, though this difference should be interpreted cautiously given the low human–LLM correlation on Clarity ($r = 0.100$). **Gemini-2.5-Flash** trails both competitors (3.355), primarily due to lower Identification (3.931) and Impact (2.750) scores; the Identification gap is the most reliable indicator of its weaker performance.

Effect of Knowledge Distillation via Fine-Tuning. The three fine-tuned 7–8B models achieve overall averages of 3.711–3.751, closely approaching the commercial API models (3.355–3.822) despite having substantially fewer parameters. This demonstrates that LoRA-based knowledge distillation from commercial model outputs effectively transfers explanation capabilities to compact open-source models. Notably, **Qwen2.5-7B-FT** achieves an Identification score of 4.265—surpassing Gemini-2.5-Flash (3.931) and approaching Claude-3.5-Haiku (4.137)—while its base counterpart Qwen2.5-7B scores only 2.813 on the same dimension. This represents a **+1.452 point improvement** (+51.6%) from fine-tuning, confirming that domain-specific adaptation fundamentally transforms a model's ability to identify and characterise vulnerability patterns.

The fine-tuning effect is equally pronounced for Llama-3.1-8B: the base model averages 2.444, while the fine-tuned variant reaches 3.750 (+1.306, +53.4% improvement). These gains appear across all three dimensions,

Table 23. Explanation quality benchmark (mean across three evaluator judges). Scores are on a 1–5 scale. **Bold** indicates the highest score per column. Models are grouped by category: commercial API, fine-tuned open-source, and base open-source.

Model	Average	Clarity	Identification	Impact
Commercial API Models				
GPT-4o	3.822	3.995	4.351	3.118
Claude-3.5-Haiku	3.774	4.022	4.137	3.162
Gemini-2.5-Flash	3.355	3.383	3.931	2.750
Fine-Tuned Open-Source Models (7–8B)				
Llama-3.1-8B-Instruct-FT (ours)	3.750	3.951	4.126	2.861
Qwen2.5-7B-FT (ours)	3.751	3.948	4.265	2.943
Qwen2.5-Coder-7B-FT (ours)	3.711	3.857	4.155	2.956
Open-Source Base Models				
GPT-OSS-20B	**3.968**	**4.113**	**4.421**	**3.244**
DeepSeek-Coder-V2-16B	2.655	2.696	2.932	2.188
DeepSeek-R1-8B[†]	2.819	2.896	3.436	2.153
Qwen2.5-7B	2.615	2.692	2.813	2.166
Llama-3.1-8B	2.444	2.431	2.671	2.088

[†]Only 24 of 197 explanations were non-empty due to model's inability to output in correct format; scores reflect this reduced sample.

Table 24. Per-evaluator judge scores (overall average) for each explanation model. Columns report the mean overall score assigned by each judge independently.

Model	GPT-4o	Claude-3.5-Haiku	Gemini-2.5-Flash
GPT-4o	4.092	3.195	4.178
Claude-3.5-Haiku	3.205	3.597	4.520
Gemini-2.5-Flash	3.467	3.015	3.583
GPT-OSS-20B	3.892	3.575	4.437
Llama-3.1-8B-Instruct-FT	3.760	3.248	4.241
Qwen2.5-7B-FT	3.727	3.277	4.248
Qwen2.5-Coder-7B-FT	3.680	3.288	4.166
DeepSeek-Coder-V2-16B	2.683	2.341	2.942
DeepSeek-R1-8B	2.602	2.611	3.243
Qwen2.5-7B	2.630	2.303	2.911
Llama-3.1-8B	2.505	2.215	2.612

Just Accepted

though the improvement is most confidently established on Identification ($r = 0.860$); the gains on Clarity and Impact, while directionally consistent, are measured by dimensions with lower human–LLM agreement (Section 4.3.1) and should therefore be interpreted as indicative rather than conclusive. Importantly, the fine-tuned models achieve these scores with models small enough (7–8B parameters) for deployment in resource-constrained environments such as CI/CD pipelines and IDE plugins, where commercial API access may be unavailable or cost-prohibitive.

Open-Source Base Models. Among the base (non-fine-tuned) open-source models, **GPT-OSS-20B** stands out as the top performer (3.968), exceeding all commercial API models in every dimension—most notably on Identification (4.421 vs. GPT-4o's 4.351), the dimension with the highest human–LLM agreement. Its larger parameter count (20B) and stronger pre-training provide sufficient capacity for generating high-quality explanations without task-specific fine-tuning. The remaining base models—DeepSeek-Coder-V2-16B (2.655), Qwen2.5-7B (2.615), and Llama-3.1-8B (2.444)—cluster in the 2.4–2.7 range, reflecting the limited ability of smaller pre-trained models to simultaneously balance code comprehension, vulnerability reasoning, and natural language generation without domain adaptation. **DeepSeek-R1-8B** presents an anomalous case: it produced non-empty explanations for only 24 of 197 functions (12.2%), yielding a modest average of 2.819 on the reduced sample. The high failure rate suggests that reasoning-oriented models not specifically designed for code generation may struggle with the structured output format required by the explanation task.

Inter-Judge Variability. Table 24 reveals systematic differences among evaluator judges. The Gemini-2.5-Flash evaluator consistently assigns the highest scores (mean across all models: 3.644), while the Claude-3.5-Haiku evaluator assigns the lowest (mean: 2.970), and the GPT-4o evaluator falls in between (mean: 3.340). Despite this level shift, the *relative ranking* of explanation models remains stable across all three judges: the Kendall's τ rank correlation between any pair of judges exceeds 0.90, indicating strong agreement on which models produce superior explanations. This consistency validates our multi-judge protocol: while individual judges exhibit different calibration baselines, the consensus ranking is robust and not dominated by any single evaluator's bias.

Dimension-Level Analysis. Across all models, **Identification** consistently receives the highest scores (mean: 3.658), followed by **Clarity** (mean: 3.453) and **Impact** (mean: 2.693). Importantly, this ranking aligns with the reliability hierarchy established in Section 4.3.1: Identification—the highest-scored dimension—is also the most reliably assessed by LLM judges ($r = 0.860$), lending strong confidence to cross-model comparisons on this axis. Impact—the lowest-scored dimension—has only moderate human–LLM agreement ($r = 0.321$), meaning that the absolute score levels should be interpreted with caution; however, relative *rankings* among models on Impact are likely meaningful, since all three independent judges agree on the ordering (Kendall's $\tau > 0.90$ between judge pairs). Clarity exhibits near-zero human–LLM correlation ($r = 0.100$), so model differences on this dimension are the least trustworthy at the individual-score level, though the multi-judge consensus ranking may still capture genuine quality differences. The systematically lower Impact scores suggest that articulating the real-world consequences of vulnerabilities—including realistic attack scenarios, quantified financial risks, and context-aware mitigations—is the most challenging aspect of explanation generation. This pattern holds for both commercial and open-source models, indicating a fundamental difficulty: accurate impact assessment requires not only code-level understanding but also broader knowledge of DeFi protocols, economic incentive structures, and historical exploit patterns that may be underrepresented in pre-training corpora. The Impact dimension therefore represents the primary opportunity for improving explanation quality in future work, potentially through retrieval-augmented generation (RAG) with curated exploit databases or post-hoc enrichment with real-world incident data.

Key Findings. Five conclusions emerge from this benchmark, interpreted in light of the per-dimension judge reliability analysis (Section 4.3.1). First, **knowledge distillation via fine-tuning closes the gap with commercial models**: fine-tuned 7–8B models achieve 97–98% of GPT-4o's overall average score—and, on the most reliable dimension (Identification), reach 95–98% of GPT-4o's level (4.126–4.265 vs. 4.351)—while being deployable on consumer-grade hardware. Second, **model scale alone is insufficient**: without fine-tuning, 7–16B models score 30–36% below commercial APIs, whereas fine-tuning lifts them to near parity. Third, on the **most reliable evaluation dimension (Identification, $r = 0.860$)**, the fine-tuned Qwen2.5-7B-FT (4.265) surpasses Gemini-2.5-Flash (3.931) and approaches Claude-3.5-Haiku (4.137), demonstrating that domain-specific fine-tuning can match or exceed commercial-model technical accuracy—a finding grounded in the evaluation dimension with the strongest

Just Accepted

human–LLM agreement. Fourth, **Impact assessment is the hardest dimension** for all model classes, pointing to a structural limitation of current LLMs in contextualising code-level findings within broader security and economic frameworks; however, the moderate judge reliability on this dimension ($r = 0.321$) means that absolute Impact scores should be interpreted with caution. Fifth, the **multi-judge protocol yields robust rankings**: despite individual calibration differences, all three judges agree on the relative quality ordering (Kendall's $\tau > 0.90$ between judge pairs), lending confidence to the benchmark conclusions even for the dimensions with lower absolute human–LLM agreement.

5 Threats to Validity

We organise threats to validity along the four standard dimensions: construct validity, internal validity, external validity, and conclusion validity.

Construct Validity. Our framework operates on Solidity source code and analyses contracts at the AST level. Consequently, it targets source-code-level vulnerability patterns—such as missing guard checks, unsafe external calls, and arithmetic overflows—that are identifiable from syntactic and control-/data-flow structure alone. Vulnerabilities that depend on *on-chain runtime state* (e.g., specific storage slot values), *execution sequences spanning multiple transactions* (e.g., complex multi-step oracle manipulation), or *environment-dependent conditions* (e.g., gas-price dynamics, mempool ordering) fall outside the scope of static source-code analysis and are not captured by our model. This limitation is shared by all existing static and ML-based smart contract vulnerability detectors, none of which model on-chain execution semantics. Tools that address runtime-dependent vulnerabilities, such as Sereum [Rodler et al. 2018], operate at the EVM bytecode/execution level and are complementary to source-level approaches like ours.

Importantly, HCA-SGNN *does* capture a significant class of vulnerabilities whose exploitability depends on invocation sequences across *multiple functions within a single contract*. The inter-function cross-attention module allows each function's embedding to attend to all other functions, enabling the model to learn coupled vulnerability patterns such as reentrancy (where an external call in one function re-enters another before state is updated), transaction order dependence (where a setter function modifies state consumed by an executor function), and cross-function timestamp dependence. Our 23 coupled injection templates explicitly synthesise such multi-function interaction patterns, and the model achieves 96.8% contract-level recall on real-world reentrancy and 75.0% on front-running—categories that inherently require reasoning about cross-function invocation sequences. Full multi-*transaction* state-dependent vulnerabilities (e.g., flash-loan attacks requiring atomic cross-contract call chains) remain out of scope. Extending HCA-SGNN with symbolic execution traces or transaction-level features to capture such state-dependent vulnerabilities represents a promising direction for future work.

A further construct-validity concern arises from the binary classification formulation ($y_i \in \{0, 1\}$). When a single function contains multiple co-occurring vulnerabilities of different types (e.g., a reentrancy and an unchecked return value), the model can only report that the function is vulnerable—it cannot identify how many or what types of issues co-exist. Correspondingly, the explanation pipeline generates a single explanation per flagged function, potentially describing only the most salient vulnerability. This limitation stems from the single-injection training policy (which ensures unambiguous labels but precludes multi-vulnerability training examples) and the binary classification head (which provides no gradient signal for type multiplicity). In practice, the dominant real-world case involves one vulnerability per function, so this limitation does not materially affect our reported metrics; however, it does represent an under-specification for complex, high-value contracts. Extending the classification head to a multi-label formulation and the explanation pipeline to multi-vulnerability generation are discussed as future work in Section 7.

Just Accepted

Internal Validity. A potential threat to internal validity arises from the use of synthetically injected vulnerabilities for training. Models trained on synthetic data risk learning injection-specific artifacts rather than genuine vulnerability patterns. We mitigate this risk through three mechanisms: (1) template-aware data splitting that ensures no injection template appears in more than one data partition (Section 3.2.1); (2) collision-free variable naming and version-specific adaptations that produce realistic, context-aware injections indistinguishable from developer-written code; and (3) cross-dataset evaluation on a real-world benchmark containing vulnerability types *entirely absent* from the training data (e.g., `bad_randomness`, `front_running`), on which HCA-SGNN achieves 75–87.5% recall despite zero training examples, confirming that it learns transferable structural patterns rather than template artifacts.

External Validity. Our evaluation is currently scoped to Ethereum smart contracts written in Solidity, which is the dominant smart contract language (over 90% of deployed Ethereum contracts). Although the underlying architectural principles of HCA-SGNN—subgraph decomposition, heterogeneous typed-edge graphs, dual-channel message passing—are language-agnostic, applying them to other blockchain platforms (e.g., Solana/Rust, Cardano/-Plutus, Polkadot/ink!) or smart contract languages (e.g., Vyper, Move) would require developing platform-specific AST parsers, graph construction pipelines, and injection templates. We regard cross-platform generalisation as an important direction for future work. Additionally, our injection framework currently covers 8 vulnerability categories through 59 templates. While these categories encompass the most prevalent and high-impact Solidity vulnerability classes, the fixed template set does not cover every known vulnerability type. However, HCA-SGNN's demonstrated ability to detect *unseen* vulnerability categories on the real-world benchmark—including `bad_randomness` (87.5% recall), `front_running` (75.0%), `short_addresses` (100%), and `other` (100%)—provides evidence that the model learns generalisable structural patterns (e.g., unguarded external calls, state inconsistencies, control-flow anomalies) that transfer beyond the training-time vulnerability types. Expanding the injection framework to additional vulnerability categories and employing few-shot or zero-shot transfer learning techniques would further strengthen generalisability.

Furthermore, the current framework analyses each smart contract in isolation and does not model *inter-contract* interactions. Consequently, vulnerability classes that arise from cross-contract call chains—such as cross-contract reentrancy (where a callee re-enters a different contract than the caller), flash-loan-driven composability exploits that chain multiple DeFi protocols within a single transaction, and proxy–implementation mismatches exploitable through `delegatecall` forwarding—fall outside the scope of our analysis. This limitation is shared by all existing ML-based smart contract vulnerability detectors, which uniformly operate at the single-contract level. Notably, HCA-SGNN's architecture offers a natural extension path toward inter-contract analysis: the existing inter-function cross-attention mechanism could be lifted to an inter-contract level by constructing a *contract interaction graph* whose nodes are per-contract embeddings connected by external-call edges, and the dual-channel design could be augmented with a third channel for cross-contract context. Realising this extension would additionally require a multi-contract dataset with cross-contract vulnerability labels—a resource that does not yet exist in the literature. We identify inter-contract vulnerability detection as a high-priority direction for future work.

Conclusion Validity. All reported metrics are computed on held-out test sets with template-aware splitting, and we report precision, recall, F1, accuracy, and AUC to provide a multifaceted view of performance. Confidence in our conclusions is strengthened by the consistency of results across both synthetic and real-world benchmarks, as well as by the ablation study that isolates the contribution of each architectural component. For the explanation evaluation, we mitigate the well-known self-enhancement bias of LLM-as-judge protocols [L. Zheng et al. 2023] through a multi-judge approach: three independent evaluator models (GPT-4o, Claude-3.5-Haiku, Gemini-2.5-Flash) score each explanation, and final scores are computed as the arithmetic mean. The human correlation study (Section 4.3.1) confirms that GPT-4o scores align most strongly with human judgements on the Identification

Just Accepted

dimension, while the cross-judge consistency analysis demonstrates that model rankings remain stable across all three evaluators.

6 Data and Code Availability

The injection framework source code, curated function-level dataset, real-world test set, trained HCA-SGNN model weights, baseline reimplementations, and all reproducibility scripts are publicly available at:

https://github.com/nguyenlab/sc-ast-injector (code)

https://huggingface.co/datasets/nguyenlab/sc-ast-injected (dataset)

The dataset repository includes: (1) the full sc-ast-injector output of 225,300 injected contracts covering 8 vulnerability types and 59 injection templates; (2) the curated 4,208-contract function-level dataset with template-aware train/validation/test splits and per-function vulnerability labels; (3) the 242-contract real-world test set (SmartBugs Curated ∪ SolidiFI) with function-level annotations; (4) trained HCA-SGNN model weights and baseline reimplementations; and (5) environment specifications and scripts to reproduce all reported results.

7 Conclusion

This work integrates language model–based explanation generation with graph-based learning to advance the accuracy and interpretability of smart contract vulnerability analysis. It addresses two persistent limitations of existing tools: the absence of actionable, human-readable explanations and the lack of function-level vulnerability analysis that aligns with developer mental models. The proposed framework offers a dual contribution to the usability of automated security analysis, improving both detection and interpretability.

For detection, we introduced a Hierarchical Cross-Attention Subgraph GNN (HCA-SGNN) capable of classifying vulnerabilities at the *function level*. Unlike conventional contract-level GNNs, our approach decomposes contracts into function-level subgraphs derived from control- and data-flow dependencies, enabling finer localization of vulnerabilities. Through dual-channel message passing with gated fusion and inter-function cross-attention, HCA-SGNN captures both intra-function vulnerability patterns and cross-function interactions such as reentrancy. Empirical evaluations on both synthetic and real-world datasets confirm that HCA-SGNN achieves state-of-the-art classification performance while significantly enhancing localization granularity. This function-level perspective reduces the burden of manual auditing by allowing developers to efficiently isolate vulnerable code.

To support such fine-grained analysis, we curated a large synthetic dataset of Solidity contracts annotated at the function level with structured semantic metadata. The dataset encompasses a diverse range of vulnerability types (e.g., reentrancy, arithmetic, access control, denial-of-service) and was carefully designed to ensure label fidelity and realistic injection scenarios. By providing function-level annotations, it fills a critical gap in existing benchmarks, enabling the systematic evaluation of fine-grained detection methods.

In parallel, this work advances interpretability through a pipeline for generating structured explanations driven by large language models (LLMs). Using synthetic data and Chain-of-Thought (CoT) prompting, commercial API models (GPT-4o, Claude-3.5-Haiku, Gemini-2.5-Flash) generate vulnerability explanations organised into five fields: *Bug Name*, *Affected Area*, *Description*, *Impact Assessment*, and *Mitigation*. Through LoRA-based knowledge distillation, these capabilities are transferred to compact open-source models (7–8B parameters), which achieve 97–98% of commercial model quality on average—and 95–98% on the most reliably assessed dimension (Identification)—while being deployable on consumer-grade hardware. A multi-judge evaluation protocol employing three independent LLM evaluators provides robust and bias-mitigated quality assessment across clarity, identification accuracy, and impact awareness.

Taken together, these components form an end-to-end vulnerability analysis pipeline that combines structural reasoning via HCA-SGNN with semantic interpretation via LLMs. This hybrid framework enables both precise bug detection and accessible explanation, supporting more developer-friendly smart contract auditing workflows.

Just Accepted

Several avenues remain open for future work. One limitation of the current evaluation is the size of the real-world test set: the 242-contract SmartBugs-derived corpus is, to our knowledge, one of the largest publicly available labeled real-world Solidity benchmarks that spans multiple vulnerability types with function-level ground truth, but expanding it substantially would require expert manual annotation or large-scale on-chain scraping with verified labels—tasks whose scope exceeds this work. Enlarging this corpus, particularly for underrepresented types such as front running, bad randomness, and access control, is a high-priority direction for future evaluation. On the detection side, extending HCA-SGNN to capture compositional and cross-contract vulnerabilities would improve coverage. Reformulating the classification head as a multi-label predictor ($y_i \in \{0, 1\}^K$) would enable joint detection and type identification of co-occurring vulnerabilities within a single function—addressing the current binary-label limitation that prevents the model from reasoning about vulnerability multiplicity. Complementary sub-function localization (e.g., statement-level attention or node-level vulnerability attribution) could further refine granularity, while conditioning the explanation pipeline on the predicted vulnerability-type vector rather than a single binary flag would enable the generation of distinct explanations for each co-occurring vulnerability within the same function. Evaluating closed-source LLMs (e.g., GPT-4-class models) under controlled fine-tuning conditions would further clarify the boundaries of the text-based paradigm for function-level detection; our current results with 7–8 B-parameter open-source LLMs suggest fundamental precision limitations, but substantially larger models may exhibit different scaling behaviour. On the explanation side, leveraging LLMs for interactive auditing assistants and integrating user feedback could further enhance usability. On the evaluation side, the low human–LLM correlation on Clarity ($r = 0.100$) and moderate correlation on Impact ($r = 0.321$) highlight the need for improved evaluation protocols for these inherently subjective dimensions. Promising directions include structured human-evaluation rubrics assessed by domain-expert panels, pairwise preference judgements (which mitigate the calibration issues of absolute Likert scoring), and dimension-specific judge fine-tuning on human-annotated explanation quality data. Additionally, improving explanation model performance on Impact—the lowest-scored dimension across all model classes—through retrieval-augmented generation (RAG) with curated exploit databases or post-hoc enrichment with real-world incident data represents a high-priority research direction. For deployment, adopting grammar-constrained decoding (e.g., GBNF grammars or JSON-schema-constrained generation) during explanation generation could further increase the yield of structurally well-formed outputs, complementing the post-hoc rule-based validation employed in this work. Ultimately, integrating textual and graph-based embeddings into a multimodal architecture may facilitate the joint optimization of explanation and detection.

In conclusion, this work contributes novel models, datasets, and evaluation methodologies that advance the state of the art in smart contract vulnerability detection and explanation. By bridging automated classification with human-centered understanding, it lays the groundwork for more secure and interpretable blockchain systems.

Just Accepted

Acknowledgments

This work was partly supported by Japan Science and Technology Agency (JST) as part of Adopting Sustainable Partnerships for Innovative Research Ecosystem (ASPIRE), Grant Number JPMJAP25B2.

References

Md Tauseef Alam, Raju Halder, and Abyayananda Maiti. Sept. 2024. *Detection Made Easy: Potentials of Large Language Models for Solidity Vulnerabilities.* arXiv:2409.10574 [cs.CR]. (Sept. 2024). doi:10.48550/arXiv.2409.10574.

Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. Oct. 2018. *code2vec: Learning Distributed Representations of Code.* arXiv:1803.09473 [cs]. (Oct. 2018). doi:10.48550/arXiv.1803.09473.

Anthropic. 2024. *The Claude Model Family.* Accessed: 2025-02-11. (2024). https://www.anthropic.com/claude.

Jiuyang Bu, Wenkai Li, Zongwei Li, Zeng Zhang, and Xiaoqi Li. Apr. 2025. *SmartBugBert: BERT-Enhanced Vulnerability Detection for Smart Contract Bytecode*. arXiv:2504.05002 [cs.CR]. (Apr. 2025). doi:10.48550/arXiv.2504.05002.

Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. 2020. "Measuring and Relieving the Over-Smoothing Problem for Graph Neural Networks from the Topological View." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 04. Vol. 34, 3438–3445.

ConsenSys Diligence. 2025. *Mythril: Symbolic-execution-based security analysis tool for EVM bytecode*. https://github.com/ConsenSysDiligence/mythril. Accessed: 2025-07-08. (2025).

Crytic. 2025. *Slither: Slither, the smart contract static analyzer*. https://github.com/crytic/slither. Accessed: 2025-07-08. (2025).

DeepSeek-AI et al.. June 2024. *DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence*. arXiv:2406.11931 [cs]. (June 2024). doi:10.48550/arXiv.2406.11931.

Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. June 2020. "Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts." In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. arXiv:1910.10601 [cs]. (June 2020), 530–541. doi:10.1145/3377811.3380364.

Enzyme Finance (formerly Oyente). 2025. *Oyente: An Analysis Tool for Smart Contracts*. https://github.com/enzymefinance/oyente. Accessed: 2025-07-08. (2025).

ETH SRI (Secure, Reliable, and Intelligent Systems Lab). 2025. *Securify2: Security scanner for Ethereum smart contracts supported by the Ethereum Foundation and ChainSecurity*. https://github.com/eth-sri/securify2. Accessed: 2025-07-08. (2025).

Zhangyin Feng et al.. Sept. 2020. *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. arXiv:2002.08155 [cs]. (Sept. 2020). doi:10.48550/arXiv.2002.08155.

Asem Ghaleb and Karthik Pattabiraman. July 2020. "How Effective are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection." In: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. arXiv:2005.11613 [cs]. (July 2020), 415–427. doi:10.1145/3395363.3397385.

Google DeepMind, Rohan Anil, Sebastian Borgeaud, et al.. 2024. *Gemini: A Family of Highly Capable Multimodal Models*. arXiv:2312.11805. (2024). doi:10.48550/arXiv.2312.11805.

Daya Guo, Shuo Ren, et al.. Sept. 2021. *GraphCodeBERT: Pre-training Code Representations with Data Flow*. arXiv:2009.08366 [cs]. (Sept. 2021). doi:10.48550/arXiv.2009.08366.

Daya Guo, Dejian Yang, et al.. 2025. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948. (2025). doi:10.48550/arXiv.2501.12948.

Ayush Gurjar and B. R. Chandavarkar. June 2024. "Smart Contract Vulnerabilities and Detection Methods: A Survey." In: *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, Kamand, India, (June 2024), 1–7. ISBN: 979-8-3503-7024-9. doi:10.1109/ICCCNT61001.2024.10724246.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Oct. 2021. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 [cs]. (Oct. 2021). doi:10.48550/arXiv.2106.09685.

Binyuan Hui et al.. Nov. 2024. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186 [cs]. (Nov. 2024). doi:10.48550/arXiv.2409.12186.

Gerardo Iuliano and Dario Di Nucci. Dec. 2024. *Smart Contract Vulnerabilities, Tools, and Benchmarks: An Updated Systematic Literature Review*. arXiv:2412.01719 [cs]. (Dec. 2024). doi:10.48550/arXiv.2412.01719.

Uisang Lee, Changhoon Chung, Junmo Lee, and Soo-Mook Moon. Dec. 2025. *BugSweeper: Function-Level Detection of Smart Contract Vulnerabilities Using Graph Neural Networks*. arXiv:2512.09385 [cs]. (Dec. 2025). doi:10.48550/arXiv.2512.09385.

Guohao Li, Matthias Müller, Ali Thabet, and Bernard Ghanem. 2019. "DeepGCNs: Can GCNs Go As Deep As CNNs?" In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 9267–9276.

Haochen Li, Chunjie Cao, Mengnan Wang, Zhi Li, Qi Zhang, and Jingzhang Sun. Dec. 2024. "ML-AGNN: Smart Contract Vulnerability Detection Method Based on a Multi-Level Attention Graph Neural Network." In: *2024 20th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, Harbin, China, (Dec. 2024), 431–438. ISBN: 979-8-3315-1602-4. doi:10.1109/MSN63567.2024.00066.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. May 2023. *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment*. arXiv:2303.16634 [cs]. (May 2023). Retrieved Aug. 21, 2024 from http://arxiv.org/abs/2303.16634.

Zhenguang Liu, Peng Qian, Xiaoyang Wang, Yuan Zhuang, Lin Qiu, and Xun Wang. 2021. "Combining Graph Neural Networks with Expert Knowledge for Smart Contract Vulnerability Detection." *IEEE Trans. Knowl. Data Eng.*, 1–1. arXiv:2107.11598 [cs]. doi:10.1109/TKDE.2021.3095196.

Feng Luo, Ruijie Luo, Ting Chen, Ao Qiao, Zheyuan He, Shuwei Song, Yu Jiang, and Sixing Li. Apr. 2024. "SCVHunter: Smart Contract Vulnerability Detection Based on Heterogeneous Graph Attention Network." en. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, Lisbon Portugal, (Apr. 2024), 1–13. ISBN: 979-8-4007-0217-4. doi:10.1145/3597503.3639213.

Meta AI. 2024. *Meta-Llama-3.1-8B-Instruct*. https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct. Accessed: 2024-07-23. (2024).

Christensen Choong Ming Jie, Daveena Sri A-P Michael Rajah, Gyvayhni A-P Ganisen, Tharanitharan A-L Chandran, Ahmad Sahban Rafsanjani, and Muhammed Basheer Jasser. June 2024. "A Secure Data Sharing Platform in the Era of Web 3.0 by Leveraging the Power of Blockchain Technology." In: *2024 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS)*. IEEE, Shah Alam, Malaysia, (June 2024), 291–296. ISBN: 979-8-3503-7210-6. doi:10.1109/I2CACIS61270.2024.10649818.

Just Accepted

OpenAI. 2024. *GPT-4o System Card*. Accessed: 2025-02-11. (2024). https://openai.com/index/gpt-4o-system-card/.

Samuel Banning Osei, Rubing Huang, and Zhongchen Ma. May 2025. "An Attention-based Wide and Deep Neural Network for Reentrancy Vulnerability Detection in Smart Contracts." en. *Journal of Systems and Software*, 223, (May 2025), 112361. doi:10.1016/j.jss.2025.112361.

Qwen et al.. Jan. 2025. *Qwen2.5 Technical Report*. arXiv:2412.15115 [cs]. (Jan. 2025). doi:10.48550/arXiv.2412.15115.

Michael Rodler, Wenting Li, Ghassan O. Karame, and Lucas Davi. Dec. 2018. *Sereum: Protecting Existing Smart Contracts Against Re-Entrancy Attacks*. arXiv:1812.05934 [cs]. (Dec. 2018). doi:10.48550/arXiv.1812.05934.

Martina Rossini. 2022. *slither-audited-smart-contracts*. https://huggingface.co/datasets/mwritescode/slither-audited-smart-contracts. Accessed: 2024-05-20. (2022).

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Feb. 2018. *Graph Attention Networks*. arXiv:1710.10903 [stat]. (Feb. 2018). doi:10.48550/arXiv.1710.10903.

Sally Junsong Wang, Jianan Yao, Kexin Pei, Hidedaki Takahashi, and Junfeng Yang. Sept. 2024. *Detecting Buggy Contracts via Smart Testing*. arXiv:2409.04597 [cs]. (Sept. 2024). doi:10.48550/arXiv.2409.04597.

Wei Wang, Jingjing Song, Guangquan Xu, Yidong Li, Hao Wang, and Chunhua Su. Apr. 2021. "ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts." *IEEE Trans. Netw. Sci. Eng.*, 8, 2, (Apr. 2021), 1133–1144. doi:10.1109/TNSE.2020.2968505.

Zexu Wang, Jiachi Chen, Peilin Zheng, Yu Zhang, Weizhe Zhang, and Zibin Zheng. Jan. 2025. "Unity is Strength: Enhancing Precision in Reentrancy Vulnerability Detection of Smart Contract Analysis Tools." *IIEEE Trans. Software Eng.*, 51, 1, (Jan. 2025), 1–13. doi:10.1109/TSE.2024.3427321.

Zhiyuan Wei, Jing Sun, Zijian Zhang, Xianhao Zhang, Xiaoxuan Yang, and Liehuang Zhu. Feb. 2025. "Survey on Quality Assurance of Smart Contracts." en. *ACM Comput. Surv.*, 57, 2, (Feb. 2025), 1–36. doi:10.1145/3695864.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-29044-2.

Chang Xu, Huaiyu Xu, Liehuang Zhu, Xiaodong Shen, and Kashif Sharif. June 2025. "Enhanced Smart Contract Vulnerability Detection via Graph Neural Networks: Achieving High Accuracy and Efficiency." *IIEEE Trans. Software Eng.*, 51, 6, (June 2025), 1854–1865. doi:10.1109/TSE.2025.3570421.

Peiyan Zhang, Yuchen Yan, Xi Zhang, Chaozhuo Li, Senzhang Wang, Feiran Huang, and Sunghun Kim. May 2024. *TransGNN: Harnessing the Collaborative Power of Transformers and Graph Neural Networks for Recommender Systems*. arXiv:2308.14355 [cs]. (May 2024). doi:10.48550/arXiv.2308.14355.

Lianmin Zheng et al.. 2023. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." In: *Advances in Neural Information Processing Systems*. Vol. 36. doi:10.48550/arXiv.2306.05685.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. June 2024. *LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models*. arXiv:2403.13372 [cs]. (June 2024). doi:10.48550/arXiv.2403.13372.

Jinyao Zhu, Xiaofei Xing, Guojun Wang, and Peiqiang Li. Nov. 2023. "Opcode Sequences-Based Smart Contract Vulnerabilities Detection Using Deep Learning." In: *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, Exeter, United Kingdom, (Nov. 2023), 284–291. ISBN: 979-8-3503-8199-3. doi:10.1109/TrustCom60117.2023.00057.

Yuan Zhuang, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. July 2020. "Smart Contract Vulnerability Detection using Graph Neural Network." en. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, Yokohama, Japan, (July 2020), 3283–3290. ISBN: 978-0-9992411-6-5. doi:10.24963/ijcai.2020/454.

Just Accepted

A Per-Template Detection Metrics

This appendix provides granular detection performance broken down by injection template category and, for reentrancy, by individual sub-template and Sereum-defined reentrancy type [Rodler et al. 2018]. All metrics are computed on the held-out synthetic test set using HCA-SGNN with the template-aware data split described in Section 4.1.

A.1 Detection by Template Category (Synthetic Test Set)

Table 25 reports contract-level and function-level precision, recall, and F1 for each injection template category. HCA-SGNN achieves perfect contract-level recall across all vulnerability categories and perfect function-level recall (1.0) for every category, with precision varying by structural complexity.

Table 25. Per-template-category detection performance on the synthetic test set. **C-Prec.**, **C-Rec.**, **C-F1**: contract-level precision, recall, F1. **F-Prec.**, **F-Rec.**, **F-F1**: function-level precision, recall, F1. **#C**: number of contracts. "Clean" row reports false-positive rates among non-vulnerable contracts.

Template Category	#C	C-Prec.	C-Rec.	C-F1	F-Prec.	F-Rec.	F-F1
Coupled (all types)	147	1.000	1.000	1.000	0.832	0.997	0.907
Point — Reentrancy	12	1.000	1.000	1.000	1.000	1.000	1.000
Point — Timestamp	34	1.000	1.000	1.000	0.773	1.000	0.872
Point — Tx Origin	31	1.000	1.000	1.000	0.861	1.000	0.925
Point — Overflow	22	1.000	1.000	1.000	0.968	1.000	0.984
Point — Unchecked Send	22	1.000	1.000	1.000	0.880	1.000	0.936
Point — Unhandled Exc.	17	1.000	1.000	1.000	1.000	1.000	1.000
Point — Underflow	15	1.000	1.000	1.000	0.833	1.000	0.909
Clean	331	—	—	—	FP: 84 C / 118 F (FPR: 25.4% / 1.5%)		

A.2 Detection by Reentrancy Sub-Template (Synthetic Test Set)

To directly address the reentrancy types identified by Sereum [Rodler et al. 2018], Table 26 provides per-sub-template detection metrics for all reentrancy templates. HCA-SGNN achieves **100% contract-level recall** across all reentrancy sub-templates, including delegated and create-based variants.

Table 26. Per-reentrancy-sub-template detection performance on the synthetic test set. Templates are grouped by Sereum reentrancy type. **C-Rec.**: contract-level recall. **F-Prec.**, **F-Rec.**, **F-F1**: function-level metrics.

Sereum Type	Sub-Template	#C	C-Rec.	F-Prec.	F-Rec.	F-F1
Same-Function	`call_value_legacy`	8	1.000	1.000	1.000	1.000
	`call_value_050`	1	1.000	1.000	1.000	1.000
	`transfer_reentrancy`	2	1.000	1.000	1.000	1.000
	`send_reentrancy`	1	1.000	1.000	1.000	1.000
Cross-Function	`coupled_reentrancy` (9 variants)	143	1.000	0.854	0.997	0.920
Delegated	`delegate_reentrancy`	2	1.000	0.813	1.000	0.897
Create-Based	`create_reentrancy`	3	1.000	0.810	1.000	0.896
All Reentrancy (19 templates)		**160**	**1.000**	**0.859**	**0.998**	**0.923**

A.3 Detection by Vulnerability Type (Real-World Test Set)

Table 27 reports per-type detection metrics on the real-world benchmark (242 contracts, 3,134 functions). Reentrancy achieves 96.8% contract-level recall (30/31), confirming that the structural patterns learned from synthetic reentrancy templates generalise effectively to organic, human-written vulnerabilities.

A.4 Reentrancy-Specific Comparison Across All Benchmarked Models

AWDNN [Osei et al. 2025] targets reentrancy exclusively, so a fair comparison requires evaluating all benchmarked models on the same reentrancy-only subset. Table 28 reports contract-level precision, recall, and F1 for each model on a reentrancy-only slice of both test sets: the synthetic slice comprises all reentrancy and cross-function (coupled

Just Accepted

Table 27. Per-vulnerability-type detection on the real-world test set. Types marked with † are *absent from synthetic training data*. **C-Prec.**, **C-Rec.**, **C-F1**: contract-level. **F-Prec.**, **F-Rec.**, **F-F1**: function-level.

Vuln. Type	#C	C-Prec.	C-Rec.	C-F1	F-Prec.	F-Rec.	F-F1
Reentrancy	31	1.000	0.968	0.984	0.538	0.903	0.675
Unhandled Exception	52	1.000	0.865	0.928	0.417	0.522	0.464
Bad Randomness†	8	1.000	0.875	0.933	0.056	0.091	0.069
Front Running†	4	1.000	0.750	0.857	0.667	0.400	0.500
DoS	6	1.000	0.667	0.800	0.600	0.500	0.546
Timestamp	5	1.000	0.600	0.750	1.000	0.500	0.667
Access Control†	17	1.000	0.529	0.692	0.211	0.211	0.211
Overflow	15	1.000	0.200	0.333	0.750	0.143	0.240
Other†	3	1.000	1.000	1.000	0.750	1.000	0.857
Short Addresses†	1	1.000	1.000	1.000	1.000	1.000	1.000
Clean	100	—	—	—	FP: 27 C / 37 F (FPR: 27.0% / 1.7%)		

reentrancy) contracts plus the full clean set (159 positive + 332 clean, 491 total); the real-world slice comprises the 31 reentrancy + 100 clean contracts (131 total). For all contract-level baselines, the saved checkpoints from the main benchmark are reused directly. For HCA-SGNN, function-level predictions are aggregated to contract level via OR (a contract is flagged as vulnerable if any of its functions is predicted vulnerable). AWDNN is re-trained on a 2,000-fragment synthetic corpus (train-split source addresses only) and evaluated on function-level fragment slices: 15 reentrancy + 15 clean for synthetic and 31 reentrancy + 31 clean for real-world; F1 values are 5-seed medians.

Table 28. Contract-level reentrancy detection across all benchmarked models. Models are sorted by real-world F1. Synthetic slice: 159 reentrancy + 332 clean (491 total); real-world slice: 31 reentrancy + 100 clean (131 total). † AWDNN (reentrancy-only detector) is evaluated on function-level fragment slices with 5-seed median F1.

	Synthetic			Real-World		
Model	**Prec.**	**Rec.**	**F1**	**Prec.**	**Rec.**	**F1**
HCA-SGNN (ours)	**0.855**	**1.000**	**0.922**	0.714	**0.968**	**0.822**
SCVHUNTER [Luo et al. 2024]	0.930	0.755	0.833	**0.833**	0.806	0.820
Hierarchical RGCN	0.563	0.899	0.693	0.411	0.968	0.577
BugSweeperLight [Lee et al. 2025]	0.521	0.925	0.667	0.271	0.613	0.376
DR-GCN [Yuan Zhuang et al. 2020]	0.551	0.925	0.690	0.154	0.258	0.193
GAT [Veličković et al. 2018]	0.636	0.836	0.723	0.035	0.032	0.033
BugSweeper [Lee et al. 2025]	0.562	0.887	0.688	0.080	0.129	0.099
TransformerGNN [P. Zhang et al. 2024]	0.586	0.811	0.681	0.054	0.065	0.059
TMP [Yuan Zhuang et al. 2020]	0.558	0.881	0.683	0.047	0.065	0.054
ML-AGNN [H. Li et al. 2024]	0.532	0.836	0.650	0.046	0.065	0.053
AWDNN† [Osei et al. 2025]	1.000	0.733	0.846	0.818	0.290	0.429

A key observation from Table 28 is that most baselines achieve their overall real-world F1 (Table 18) primarily by detecting *other* vulnerability types rather than reentrancy. Their false-positive rates on clean contracts are identical between the full-benchmark and reentrancy-only evaluations (verified), but their true-positive rates

Just Accepted

for reentrancy specifically are far lower. For example, DR-GCN detects 47 of 142 total real-world vulnerable contracts overall yet catches only 8 of 31 reentrancy contracts. Only HCA-SGNN and SCVHUNTER maintain high reentrancy-specific real-world F1 (0.822 and 0.820 respectively).

Received 20 October 2025; revised 11 May 2026; accepted 7 July 2026

Just Accepted